

Word Embeddings

Geyang Guo

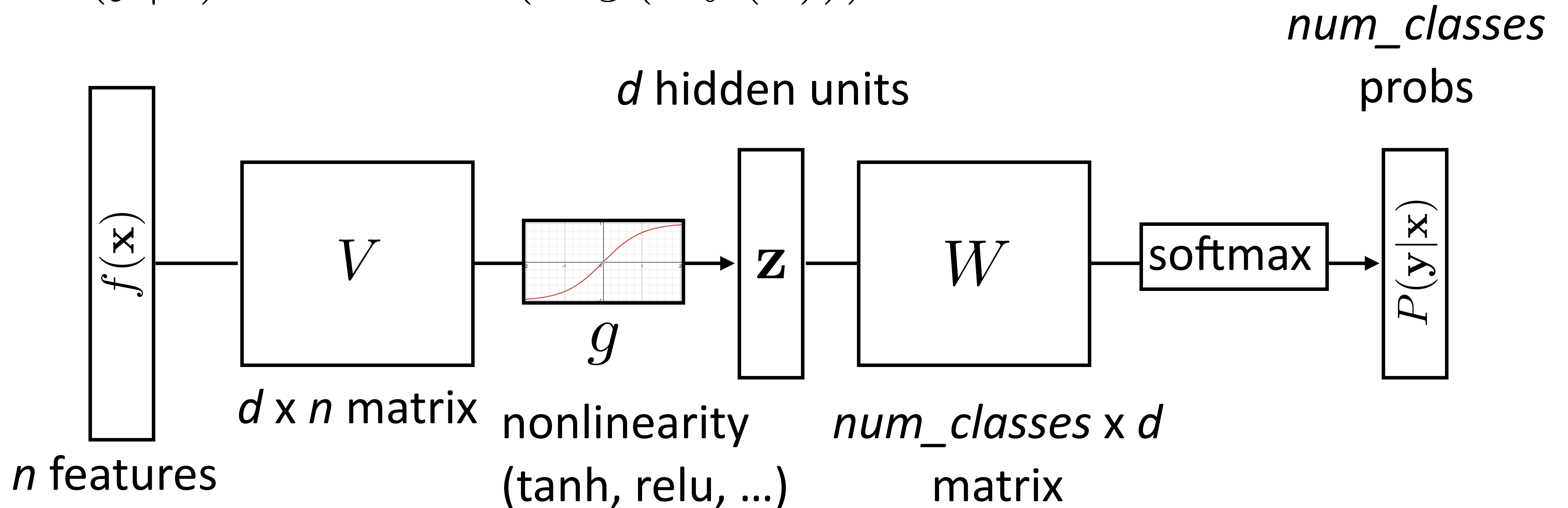
(many slides from Greg Durrett)

This Lecture

- ▶ Word representations
- ▶ word2vec/GloVe
- ▶ Reading: Eisenstein 3.3.4, 14.5, 14.6, J+M 5, Goldberg 5

Recap: Neural Networks for Classification

$$P(\mathbf{y}|\mathbf{x}) = \text{softmax}(Wg(Vf(\mathbf{x})))$$

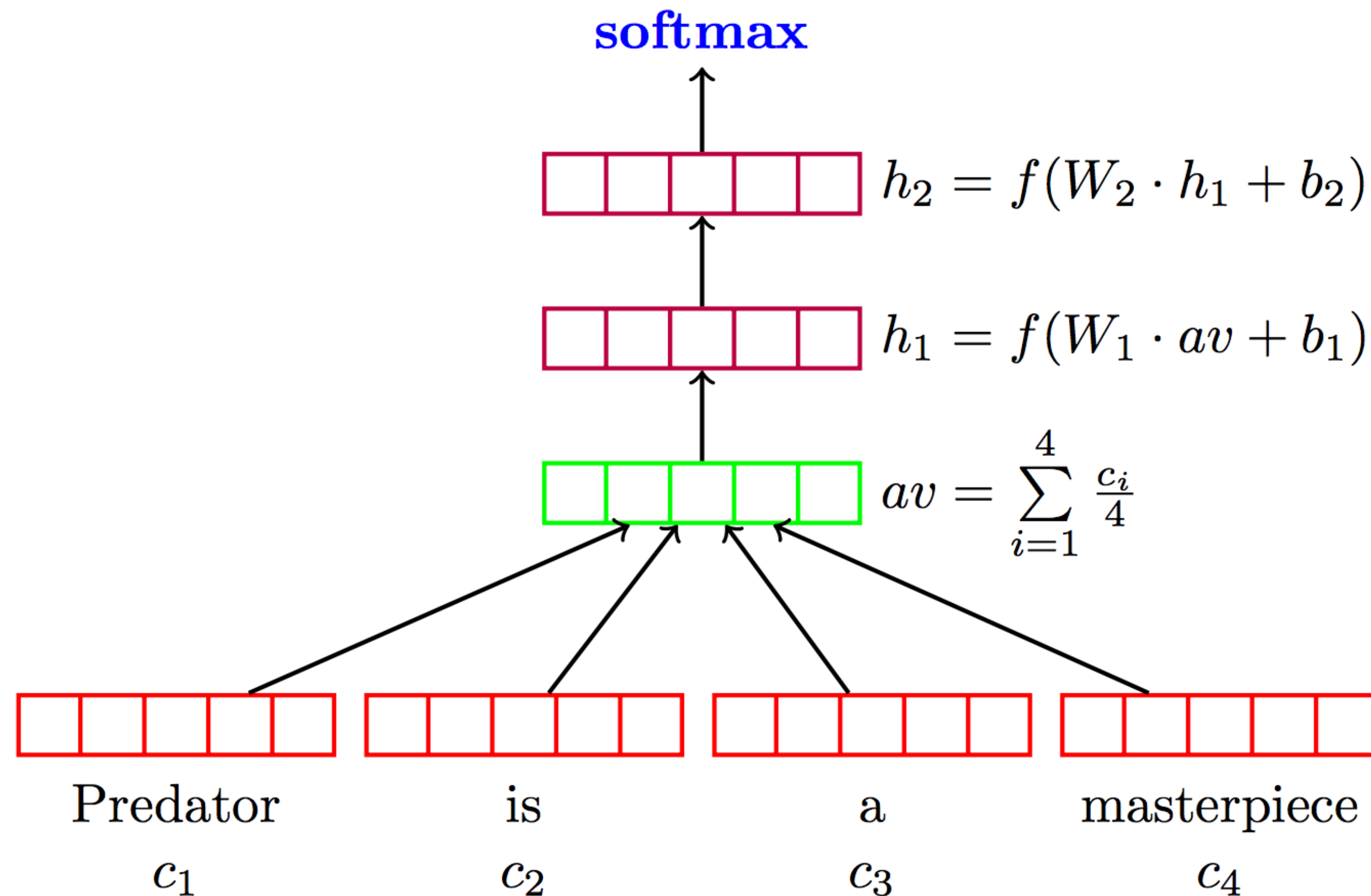


We can think of a neural network classifier with one hidden layer as building a vector $\mathbf{z}$ which is a hidden layer representation (i.e. latent features) of the input, and then running standard logistic regression on the features that the network develops in $\mathbf{z}$.

Word Representations

Sentiment Analysis

- Deep Averaging Networks: feedforward neural network on average of word embeddings from input



Word Embeddings

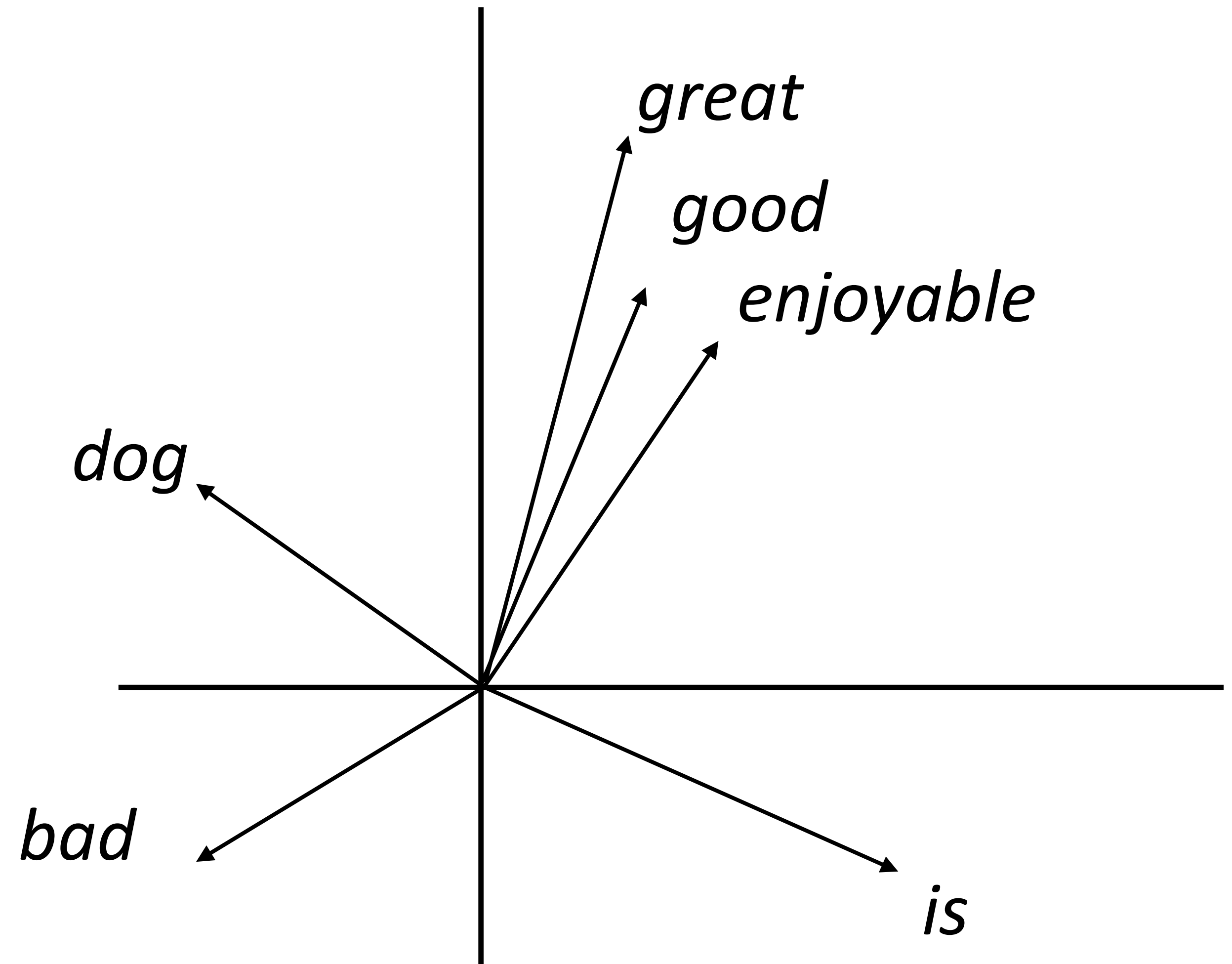
- ▶ Want a vector space where similar words have similar embeddings

the movie was great

$\approx$

the movie was good

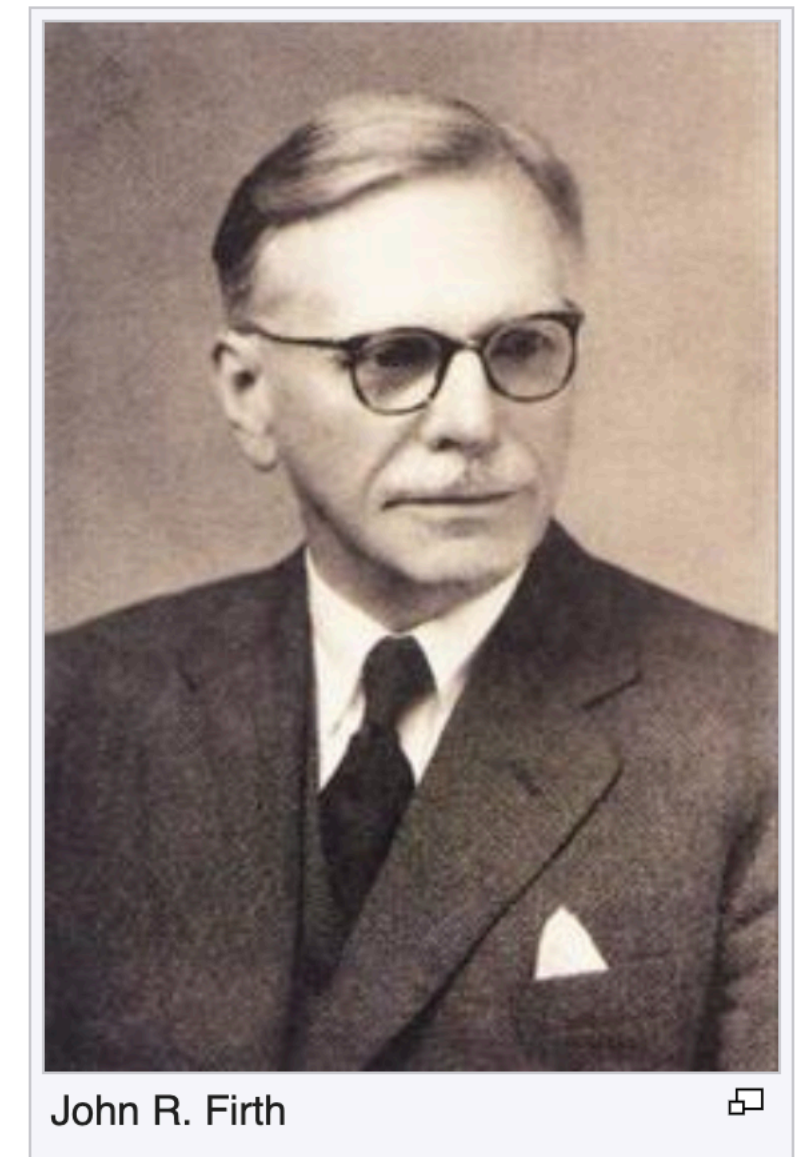
- ▶ Goal: come up with a way to produce these embeddings
- ▶ For each word, want “medium” dimensional vector (50-300 dims) representing it.



Word Representations

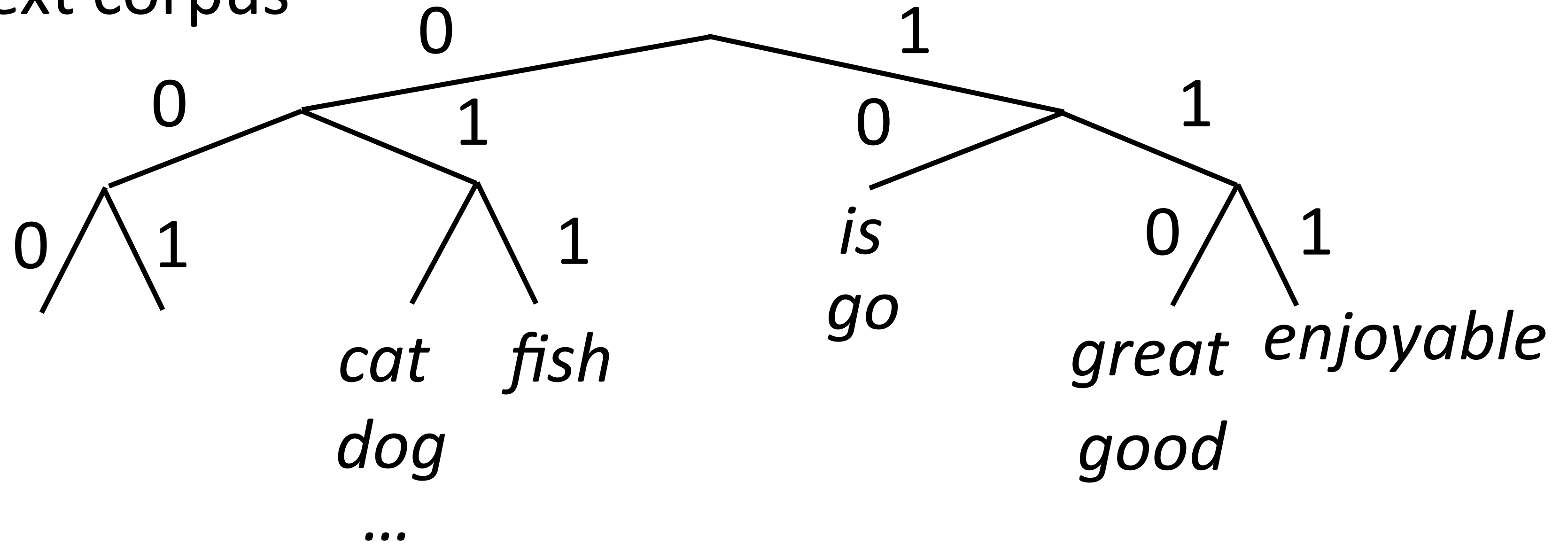
- ▶ Neural networks work very well at continuous data, but words are discrete
- ▶ Continuous model $\leftrightarrow$ expects continuous semantics from input
- ▶ “You shall know a word by the company it keeps” Firth (1957)

A bottle of ***tesgüino*** is on the table
Everybody likes ***tesgüino***
Tesgüino makes you drunk
We make ***tesgüino*** out of corn.



Discrete Word Representations

- Input: a (large) text corpus



- ▶ Maximize $P(w_i|w_{i-1}) = P(c_i|c_{i-1})P(w_i|c_i)$
- ▶ Useful features for tasks like NER, not suitable for Neural Networks
Brown et al. (1992)

Discrete Word Representations

- ▶ Brown clusters: hierarchical agglomerative *hard* clustering
- ▶ Example clusters from Miller et al. 2004

mailman	10000011010111
salesman	100000110110000
bookkeeper	1000001101100010
troubleshooter	10000011011000110
bouncer	10000011011000111
technician	1000001101100100
janitor	1000001101100101
saleswoman	1000001101100110
...	
Nike	1011011100100101011100
Maytag	10110111001001010111010
Generali	10110111001001010111011
Gap	1011011100100101011110
Harley-Davidson	10110111001001010111110
Enfield	101101110010010101111110
genus	101101110010010101111111
Microsoft	10110111001001011000
Ventritex	101101110010010110010
Tractebel	1011011100100101100110
Synopsys	1011011100100101100111
WordPerfect	1011011100100101101000
....	
John	101110010000000000
Consuelo	101110010000000001
Jeffrey	101110010000000010
Kenneth	10111001000000001100
Phillip	101110010000000011010
WILLIAM	101110010000000011011
Timothy	10111001000000001110

word cluster features (bit string prefix)

Discrete Word Representations

- ▶ Brown clusters: hierarchical agglomerative *hard* clustering
- ▶ We give a very brief sketch of the algorithm here:

- k : a hyper-parameter, sort words by frequency
- Take the top k most frequent words, put each of them in its own cluster $c_1, c_2, c_3, \dots, c_k$
- For $i = (k + 1) \dots |V|$
 - Create a new cluster c_{k+1} (*we have $k + 1$ clusters*)
 - Choose two clusters from $k + 1$ clusters based on $\text{quality}(C)$ and merge (*back to k clusters*)

$$\text{Quality}(C) = \sum_i^n \log e(w_i | C(w_i)) q(C(w_i) | C(w_{i-1})) = \sum_{c=1}^k \sum_{c'=1}^k p(c, c') \log \frac{p(c, c')}{p(c)p(c')} + G$$

mutual information
between adjacent clusters

entropy of
the word distribution

- Carry out $k - 1$ final merges (*full hierarchy*)
- Running time $O(|V|k^2 + n)$, n =#words in corpus

Word Representations

- ▶ Count-based: $tf*idf$, PPMI, ...
- ▶ Class-based: Brown Clusters, ...
- ▶ Distributed prediction-based embeddings: Word2vec (2013), GloVe (2014), FastText, ...
- ▶ Distributed contextual embeddings: ELMo (2018), BERT (2019), GPT, ...
- ▶ + many more variants: multi-sense embeddings, syntactic embeddings, ...

Neural Probabilistic Language Model

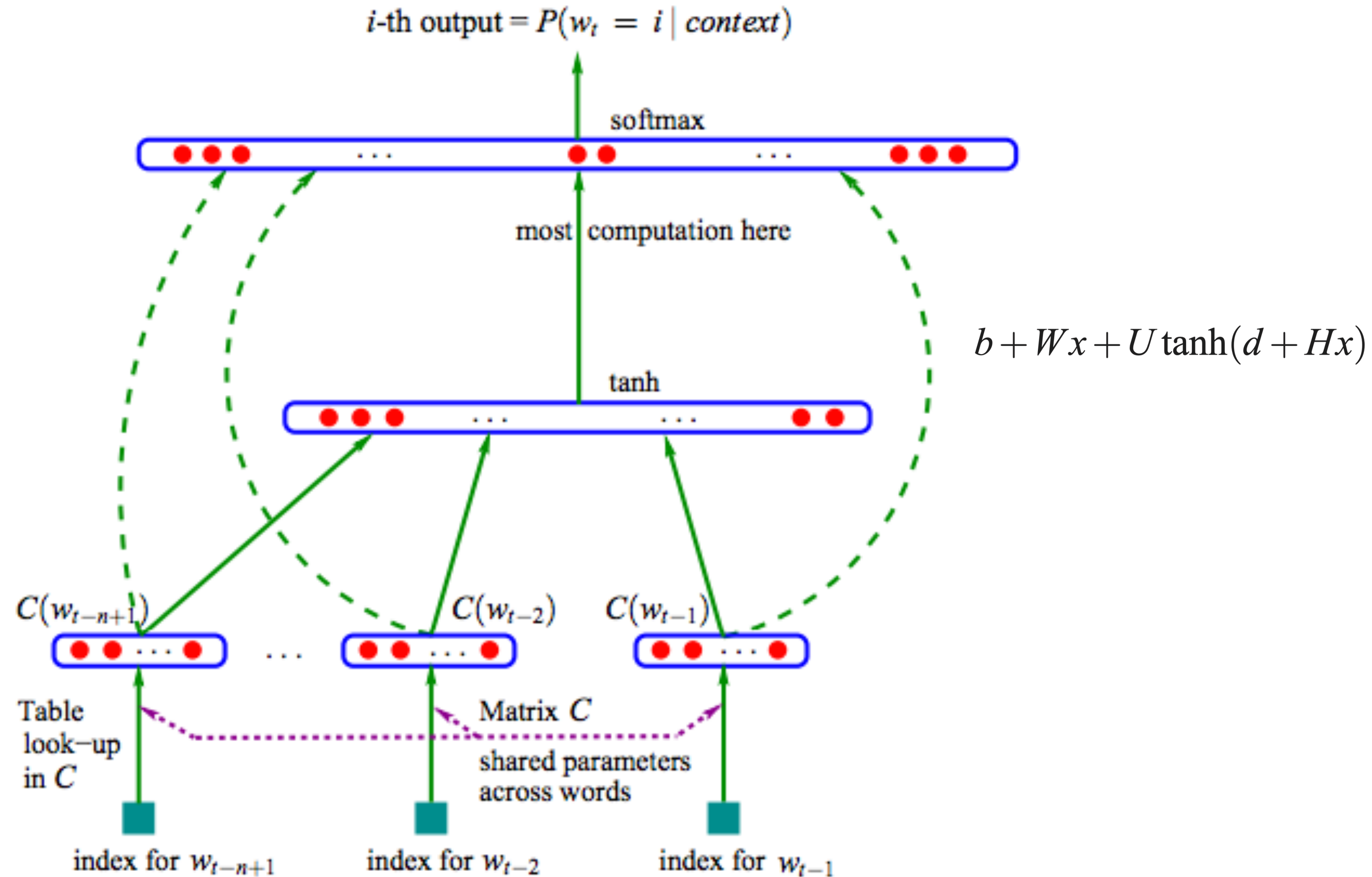


Figure 1: Neural architecture: $f(i, w_{t-1}, \dots, w_{t-n+1}) = g(i, C(w_{t-1}), \dots, C(w_{t-n+1}))$ where g is the neural network and $C(i)$ is the i -th word feature vector.

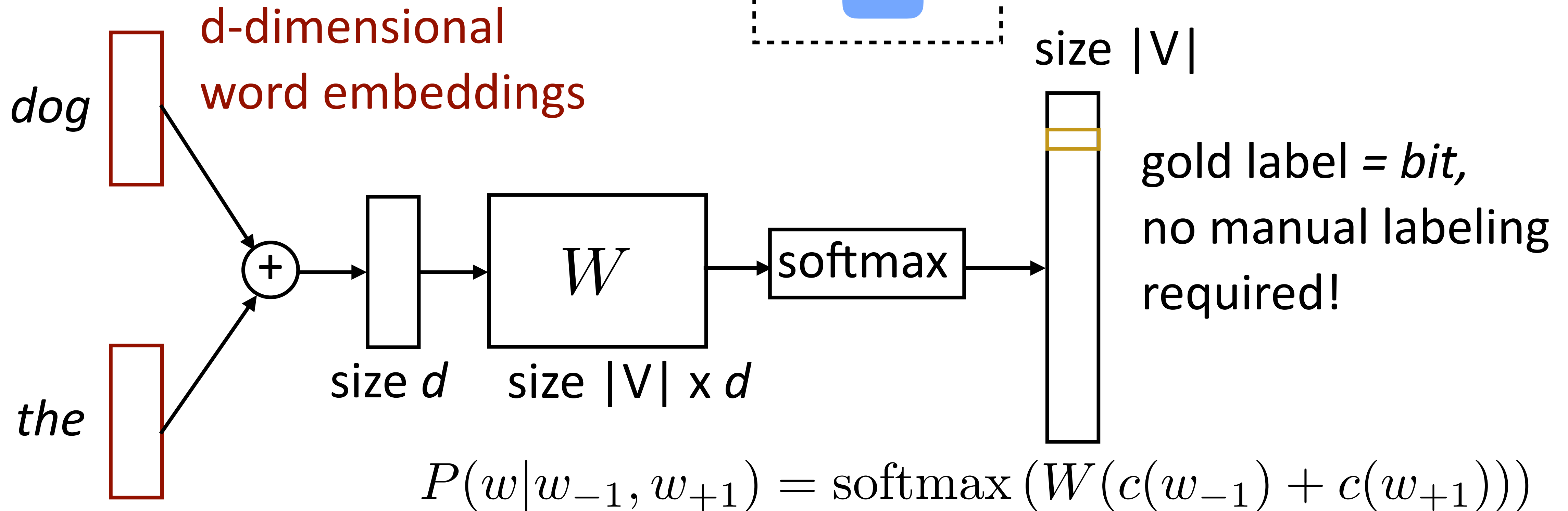
Bengio et al. (2003)

word2vec/GloVe

word2vec: Continuous Bag-of-Words

- Predict word from context

*the dog **bit** the man*



- Parameters: $d \times |V|$ (one d -length **context vector per voc word**),
 $|V| \times d$ output parameters (W)

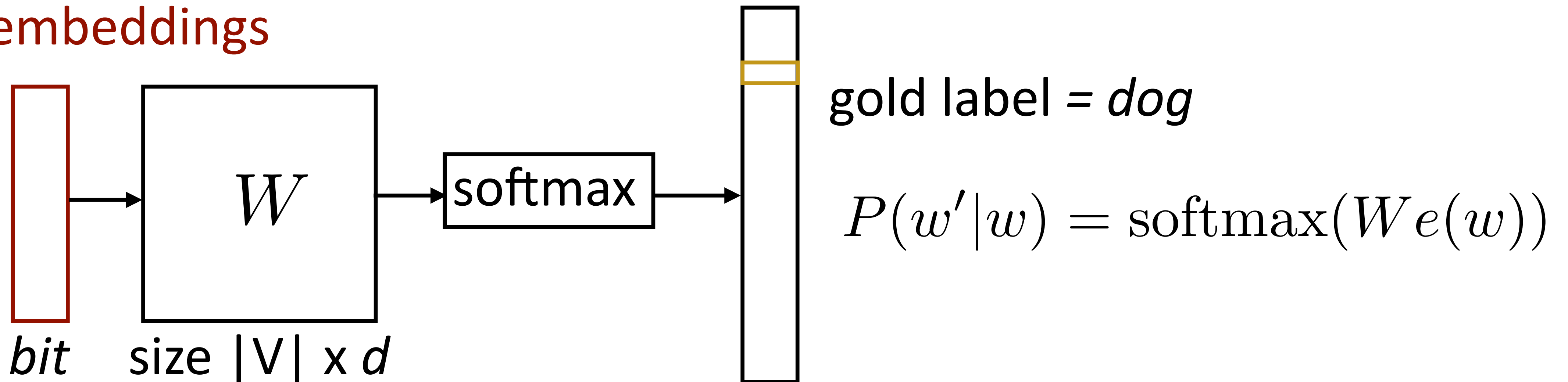
Mikolov et al. (2013)

word2vec: Skip-Gram

- Predict one word of context from word

d-dimensional
word embeddings

the dog bit the man

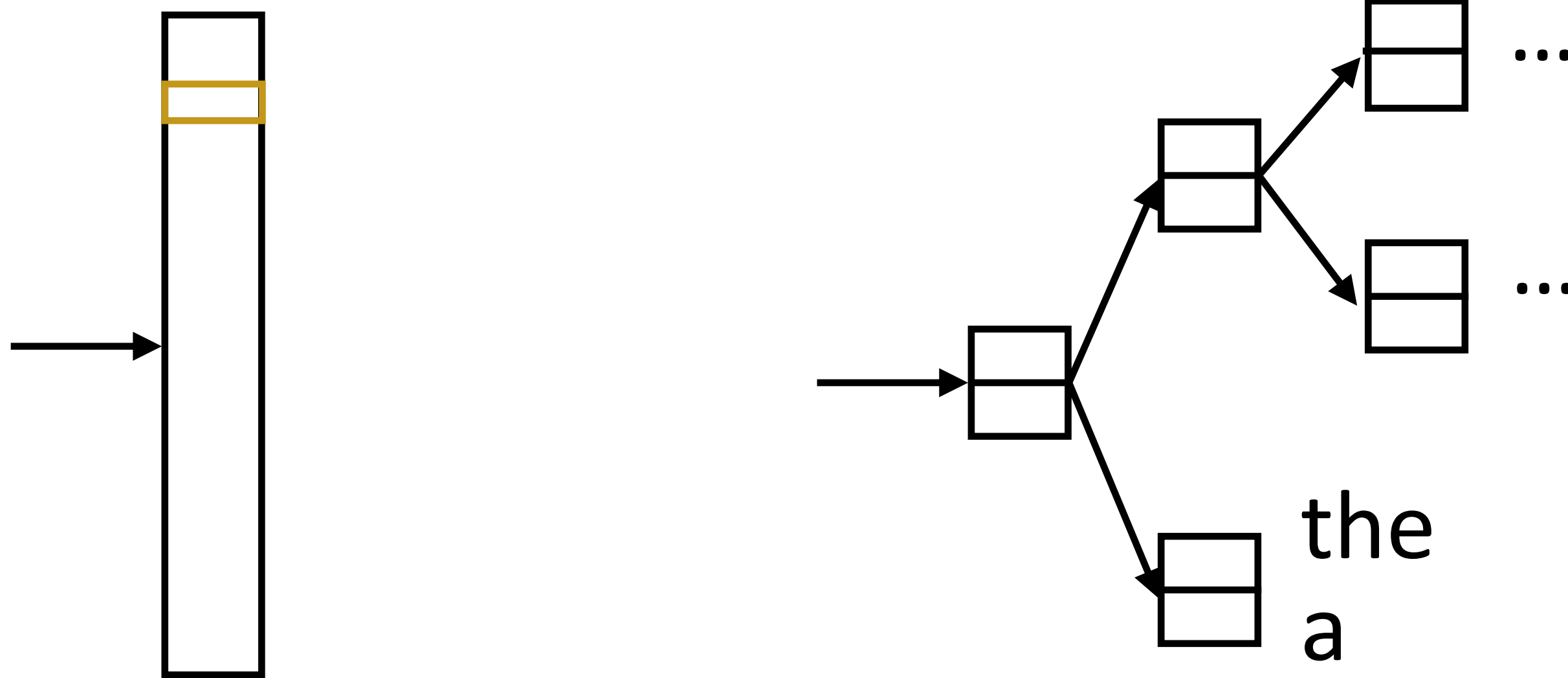


- Another training example: *bit* -> *the*
- Parameters: $d \times |V|$ **vectors**, $|V| \times d$ output parameters (W) (also usable as vectors!)

Hierarchical Softmax

$$P(w|w_{-1}, w_{+1}) = \text{softmax}(W(c(w_{-1}) + c(w_{+1}))) \quad P(w'|w) = \text{softmax}(We(w))$$

- ▶ Matmul + softmax over $|V|$ is very slow to compute for CBOW and SG



- ▶ Huffman encode vocabulary, use binary classifiers to decide which branch to take
- ▶ $\log(|V|)$ binary decisions

- ▶ Standard softmax:
 $O(|V|)$ dot products of size d
- per training instance per context word
- ▶ Hierarchical softmax:
 $O(\log(|V|))$ dot products of size d ,
Mikolov et al. 2013

Mikolov et al. (2013)

Skip-Gram with Negative Sampling

- Take (word, context) pairs and classify them as “real” or not. Create random negative examples by sampling from unigram distribution

(bit, dog) => +1

(bit, cat) => -1

(bit, a) => -1

(bit, fish) => -1

the dog bit the man

$$P(y = 1|w, c) = \frac{e^{w \cdot c}}{e^{w \cdot c} + 1}$$

words in similar contexts select for similar c vectors

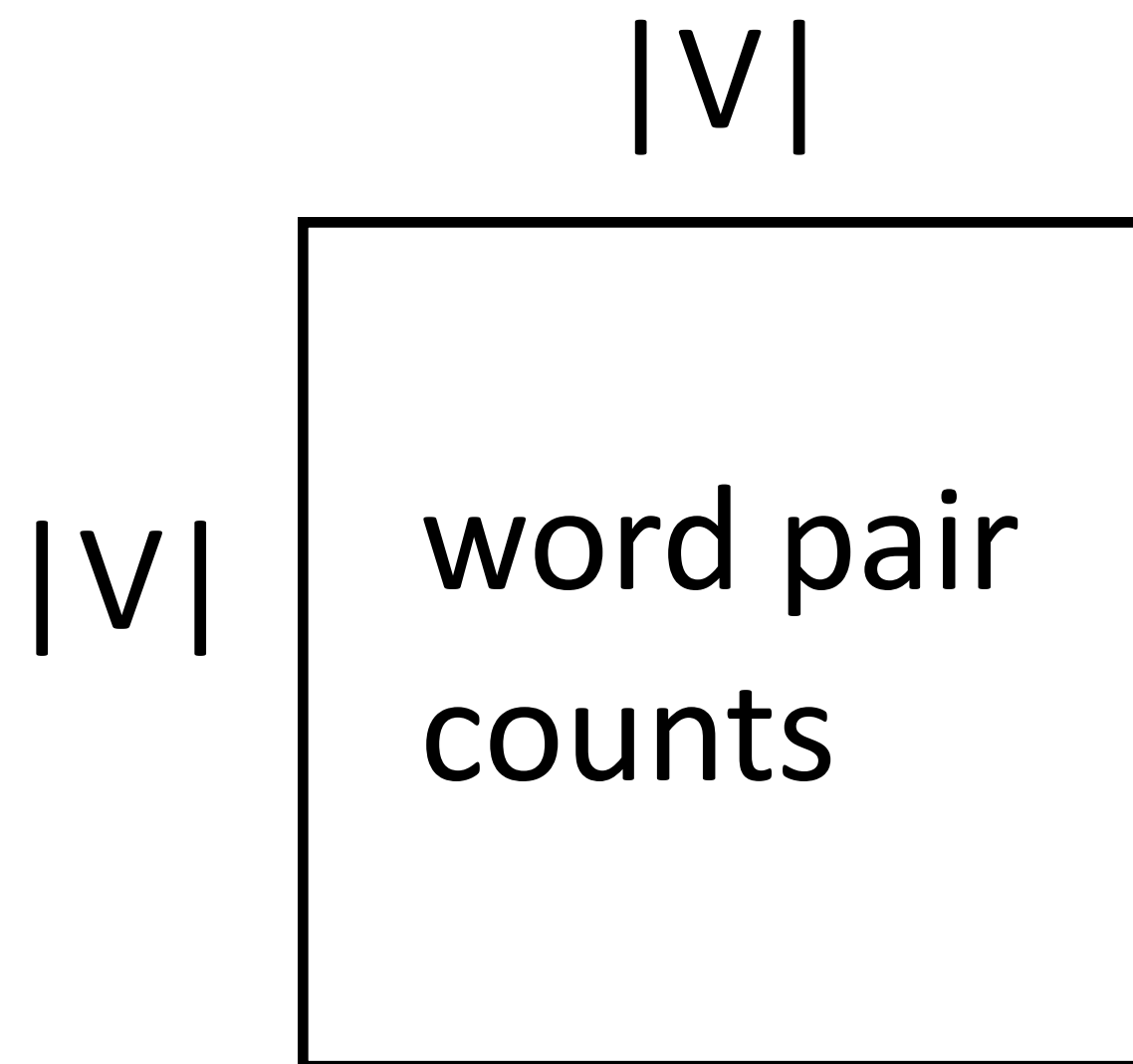
- $d \times |V|$ vectors, $d \times |V|$ context vectors (same # of params as before)

- Objective = $\log P(y = 1|w, c) + \sum_{i=1}^k \log P(y = 0|w_i, c)$ sampled

Mikolov et al. (2013)

Connections with Matrix Factorization

- ▶ Skip-gram model looks at word-word co-occurrences and produces two types of vectors



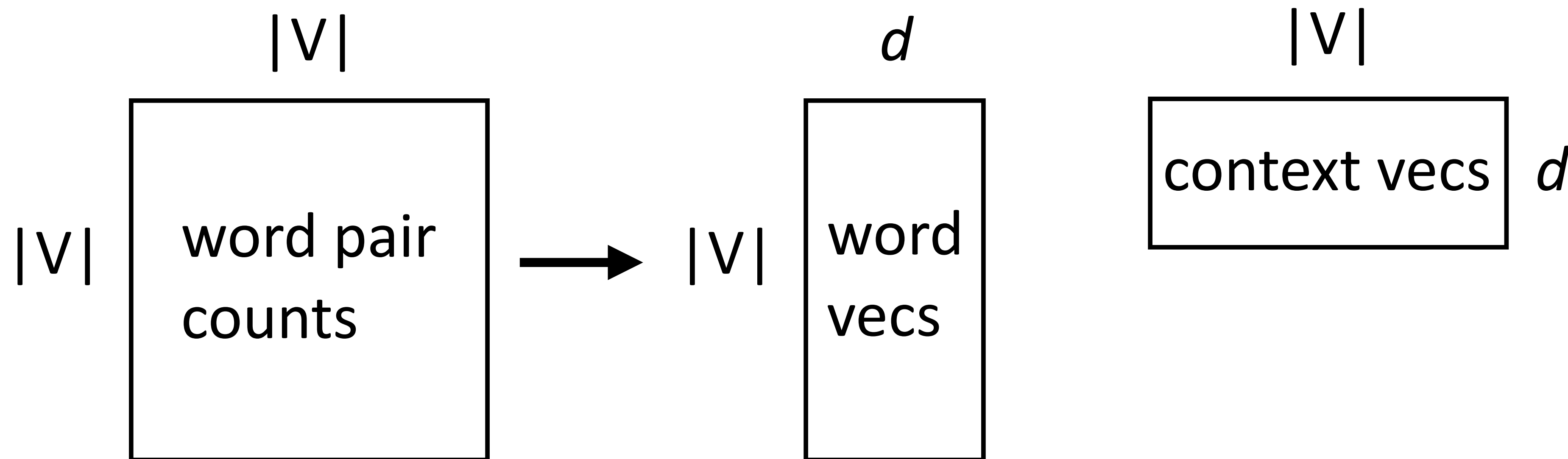
	knife	dog	sword	love	like
knife	0	1	6	5	5
dog	1	0	5	5	5
sword	6	5	0	5	5
love	5	5	5	0	5
like	5	5	5	5	2

Two words are “similar” in meaning if their context vectors are similar. Similarity == relatedness

Levy et al. (2014)

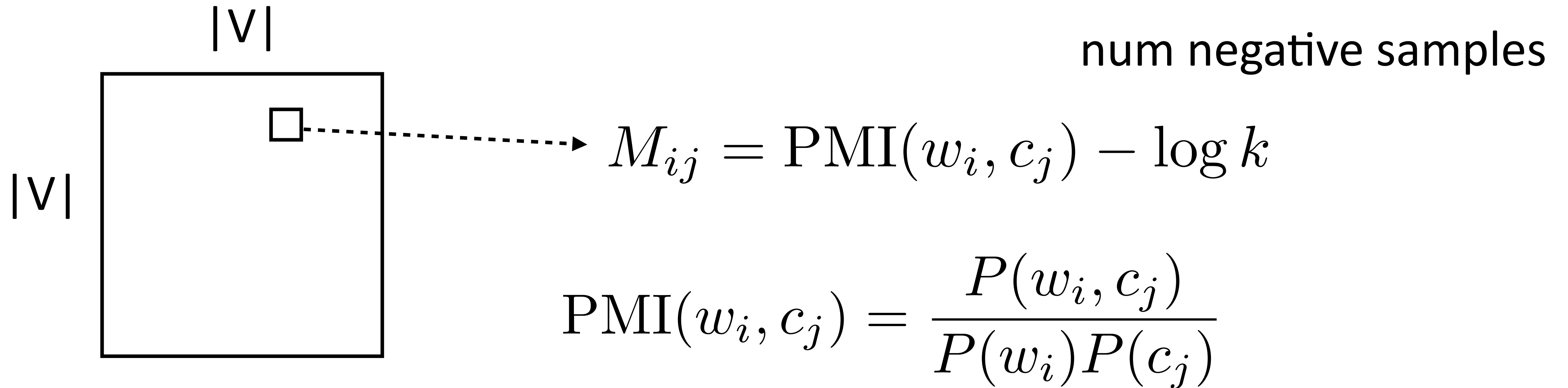
Connections with Matrix Factorization

- ▶ Skip-gram model looks at word-word co-occurrences and produces two types of vectors



- ▶ Looks almost like a matrix factorization...can we interpret it this way?

Skip-Gram as Matrix Factorization

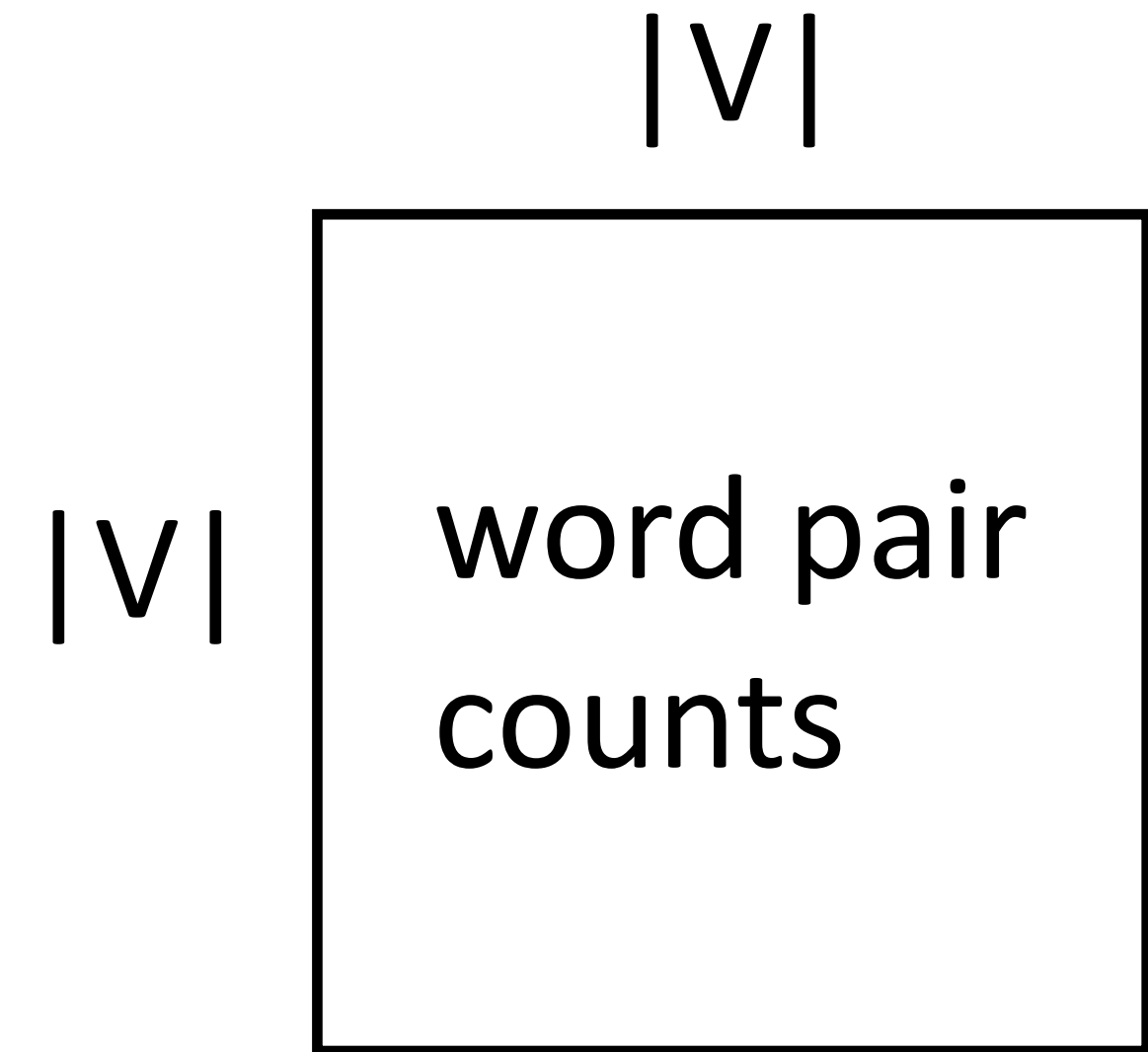


Skip-gram objective *exactly* corresponds to factoring this matrix:

- ▶ If we sample negative examples from the unigram distribution over words
- ▶ ...and it's a *weighted* factorization problem (weighted by word freq)

GloVe (Global Vectors)

- ▶ Also operates on counts matrix, weighted regression on the log co-occurrence matrix



- ▶ Objective = $\sum_{i,j} f(\text{count}(w_i, c_j)) (w_i^\top c_j + a_i + b_j - \log \text{count}(w_i, c_j))^2$
- ▶ Constant in the dataset size (just need counts), quadratic in voc size
- ▶ By far the most common non-contextual word vectors used today (30000+ citations)

Pennington et al. (2014)

Using Word Embeddings

- ▶ Approach 1 (from scratch): learn embeddings as parameters from your data
 - ▶ Often works pretty well
- ▶ Approach 2 (freeze): initialize using GloVe/word2vec/ELMo, keep fixed
 - ▶ Faster because no need to update these parameters
- ▶ Approach 3 (fine-tune): initialize using GloVe/BERT, fine-tune on your data
 - ▶ Works best for some tasks, not used for ELMo, often used for BERT

NER in Twitter

Brown clusters



2m 2ma 2mar 2mara 2maro 2marrow 2mor 2mora
2moro 2morow 2morr 2morro 2morrow 2moz 2mr
2mro 2mrrw 2mrw 2mw tmmrw tmo tmoro tmorrow
tmoz tmr tmro tmrow tmrrow tmrrw tmrw tmrww tmw
tomaro tomarow tomarro tomarrow tomm tommarow
tommarrow tommoro tommorow tommorrow
tommorw tommrow tomo tomolo tomoro tomorrow
tomorro tomorrrw tomoz tomrw tomz

Ritter et al. (2011)

Word2vec

Both

System	Fin10Dev	Rit11	Fro14	Avg
CoNLL	27.3	27.1	29.5	28.0
+ Brown	38.4	39.4	42.5	40.1
+ Vector	40.8	40.4	42.9	41.4
+ Reps	42.4	42.2	46.2	43.6
Fin10	36.7	29.0	30.4	32.0
+ Brown	59.9	53.9	56.3	56.7
+ Vector	61.5	56.4	58.4	58.8
+ Reps	64.0	58.5	60.2	60.9
CoNLL+Fin10	44.7	39.9	44.2	42.9
+ Brown	54.9	52.9	58.5	55.4
+ Vector	58.9	55.2	59.9	58.0
+ Reps	58.9	56.4	61.8	59.0
+ Weights	64.4	59.6	63.3	62.4

Table 5: Impact of our components on Twitter NER performance, as measured by F1, under 3 data scenarios.

Cherry & Guo (2015)

Visualization

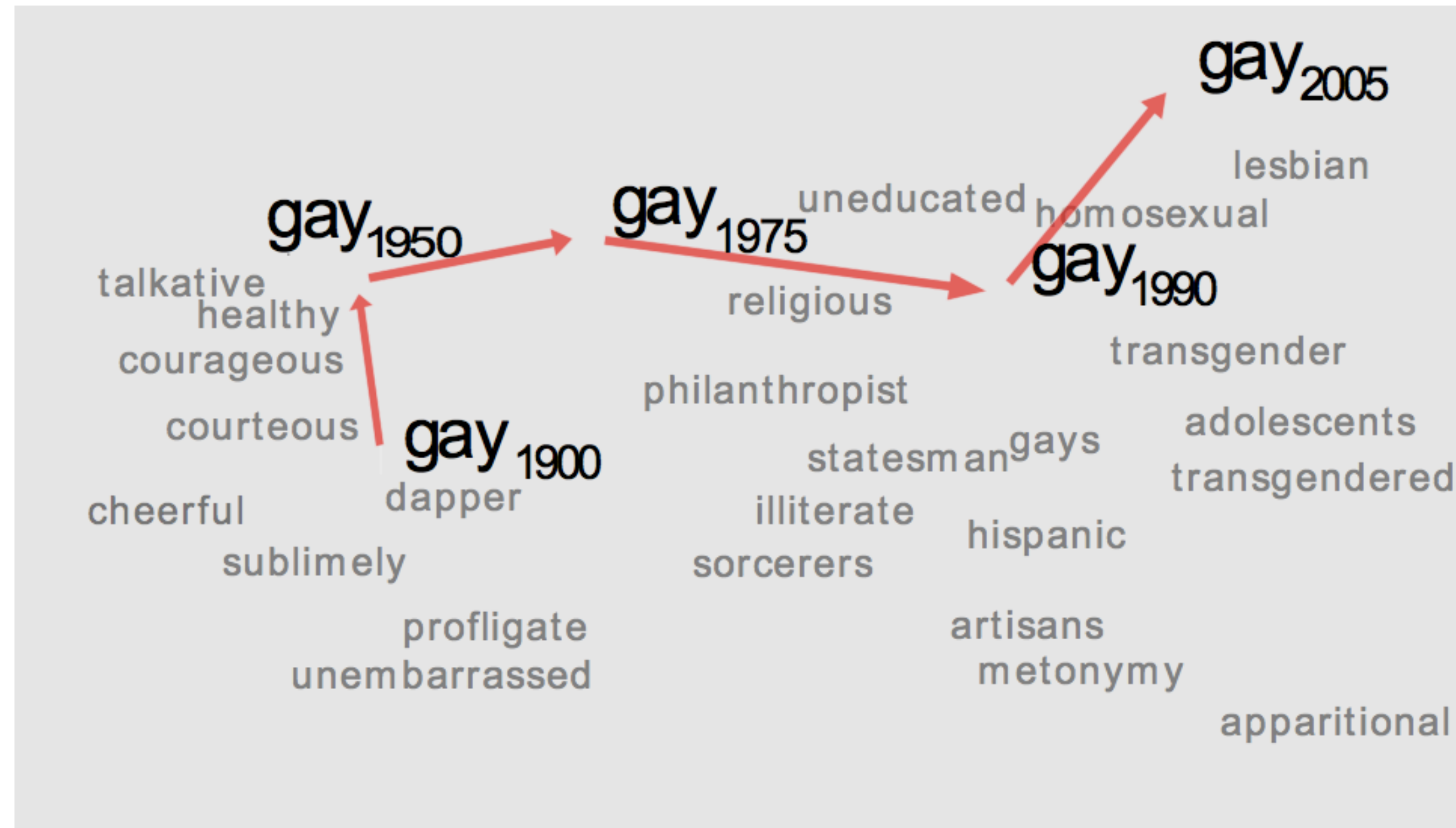


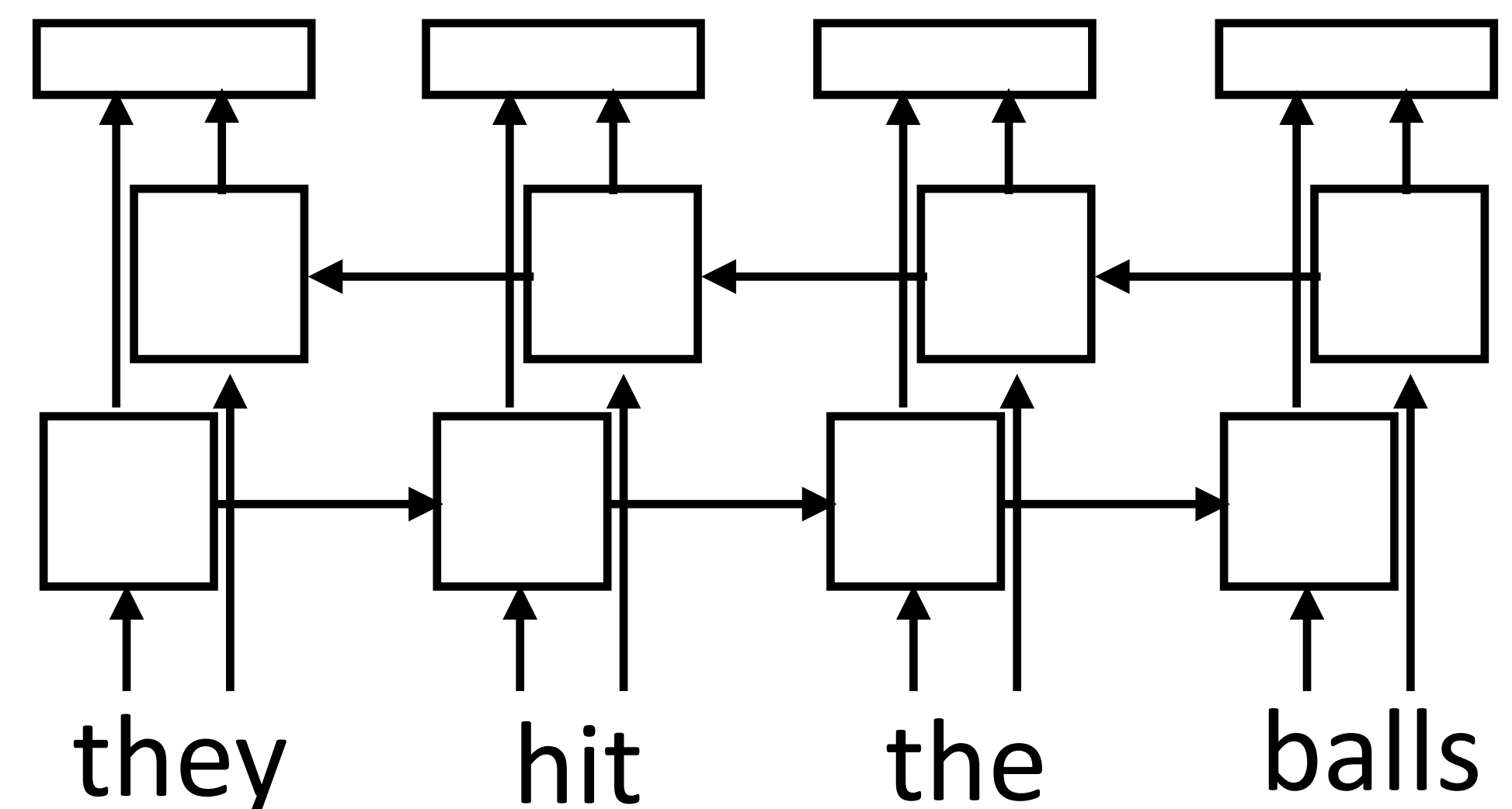
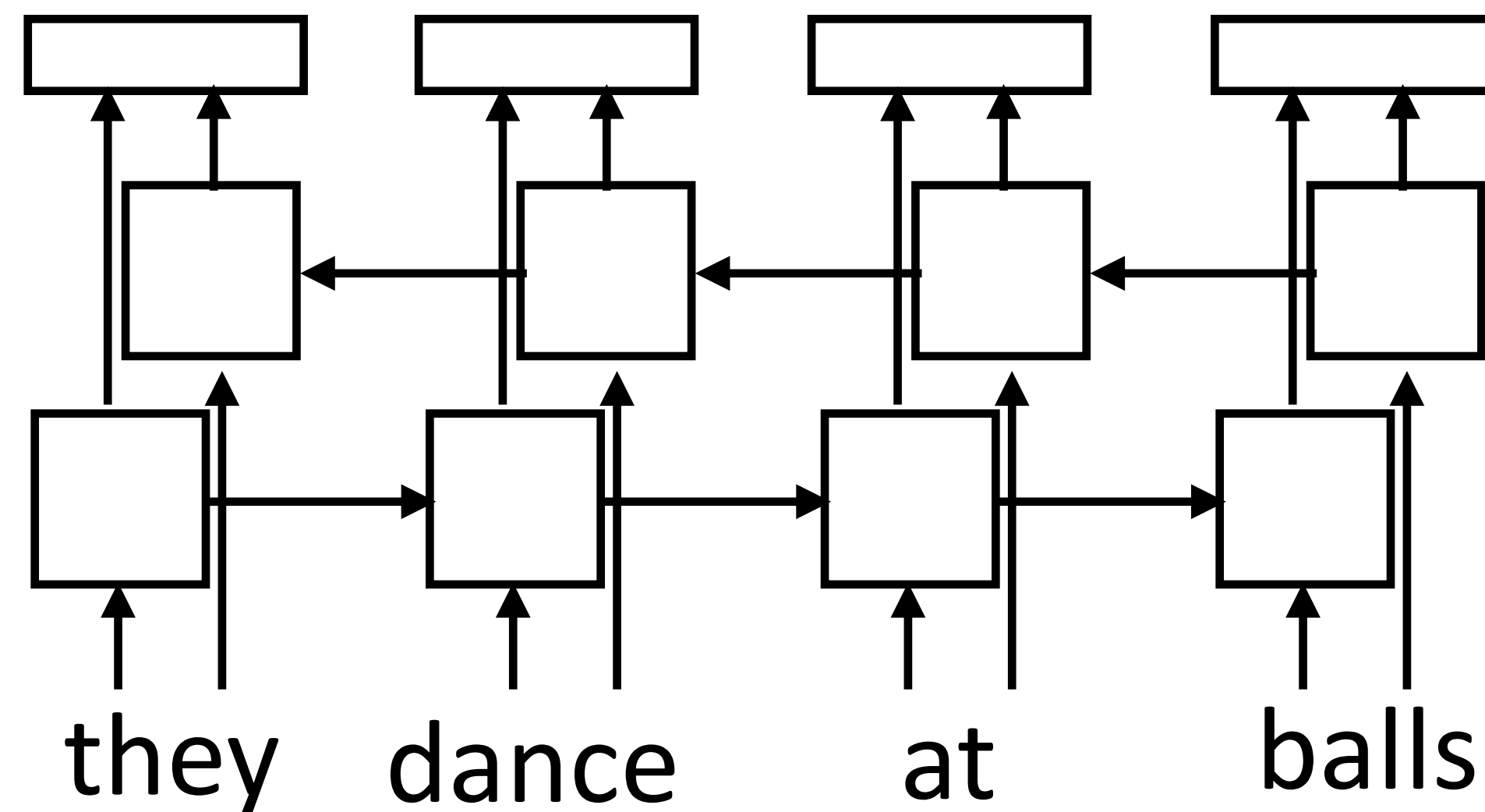
Figure 1: A 2-dimensional projection of the latent semantic space captured by our algorithm. Notice the semantic trajectory of the word **gay** transitioning meaning in the space.

Takeaways

- ▶ Word vectors: learning word $\rightarrow$ context mappings has given way to matrix factorization approaches (constant in dataset size)
- ▶ Lots of pretrained embeddings work well in practice, they capture some desirable properties
- ▶ Even better: context-sensitive word embeddings (ELMo/BERT/etc.) — will talk later in the semester
- ▶ Next time: sequence modeling, HMM, ...

Preview: Context-dependent Embeddings

- ▶ How to handle different word senses? One vector for *balls*



- ▶ Train a neural language model to predict the next word given previous words in the sentence, use its internal representations as word vectors
- ▶ *Context-sensitive* word embeddings: depend on rest of the sentence
- ▶ *Huge* improvements across nearly all NLP tasks over word2vec & GloVe

Peters et al. (2018)

Convolutional Neural Networks (CNNs)

(many slides from Greg Durrett, Stanford 23In)

Administrivia

- ▶ Reading — Goldberg 9 (CNN); Eisenstein 3.4, 7.6

A Primer on Neural Network Models for Natural Language Processing

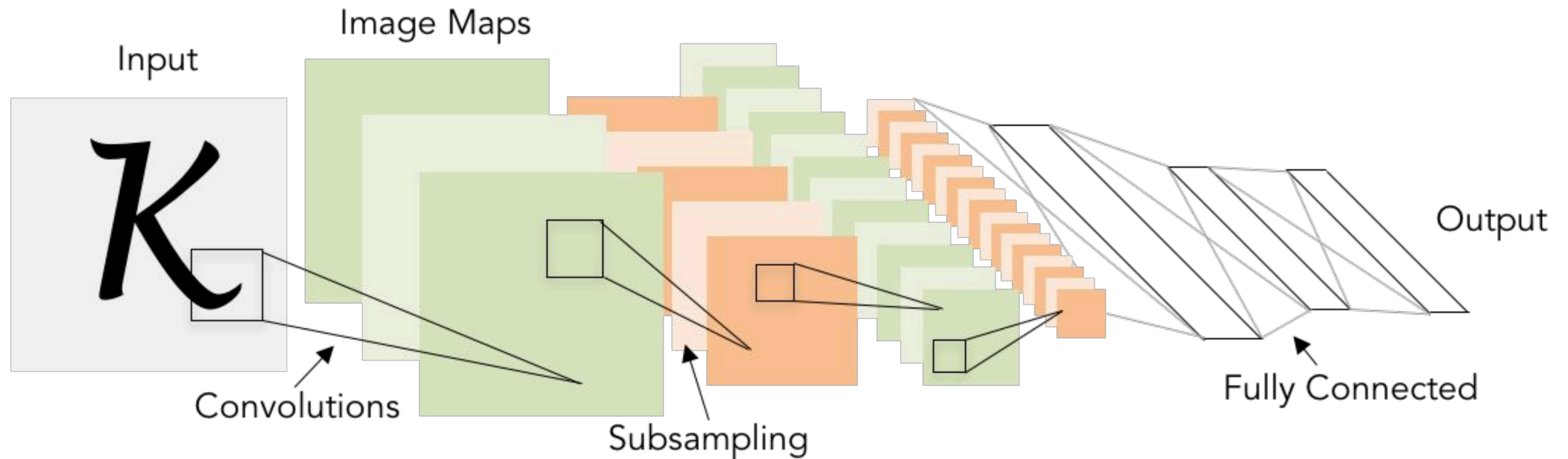
Yoav Goldberg
Draft as of October 5, 2015.

The most up-to-date version of this manuscript is available at <http://www.cs.biu.ac.il/~yogo/nnlp.pdf>. Major updates will be published on arxiv periodically. I welcome any comments you may have regarding the content and presentation. If you spot a missing reference or have relevant work you'd like to see mentioned, do let me know. first.last@gmail

Abstract

Over the past few years, neural networks have re-emerged as powerful machine-learning models, yielding state-of-the-art results in fields such as image recognition and speech processing. More recently, neural network models started to be applied also to textual natural language signals, again with very promising results. This tutorial surveys neural network models from the perspective of natural language processing research, in an attempt to bring natural-language researchers up to speed with the neural techniques. The tutorial covers input encoding for natural language tasks, feed-forward networks, convolutional networks, recurrent networks and recursive networks, as well as the computation graph abstraction for automatic gradient computation.

A Bit of History



https://www.youtube.com/watch?v=FwFduRA_L6Q

LeCun et al. (1998), earlier work in 1980s

ImageNet - Object Recognition

Steel drum

The Image Classification Challenge:

1,000 object classes

1,431,167 images



Output:

Scale

T-shirt

Steel drum

Drumstick

Mud turtle



Output:

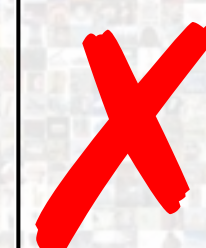
Scale

T-shirt

Giant panda

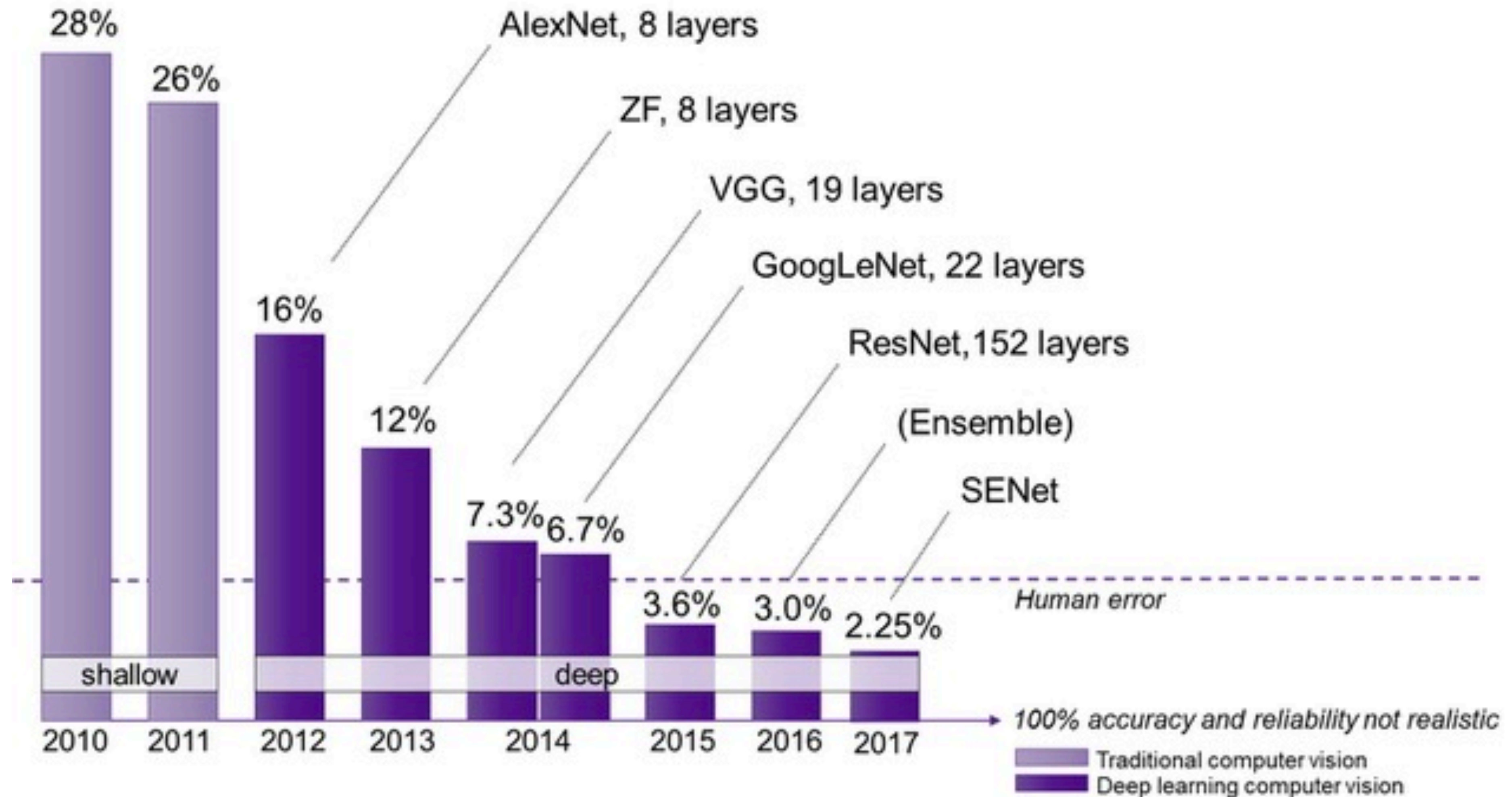
Drumstick

Mud turtle



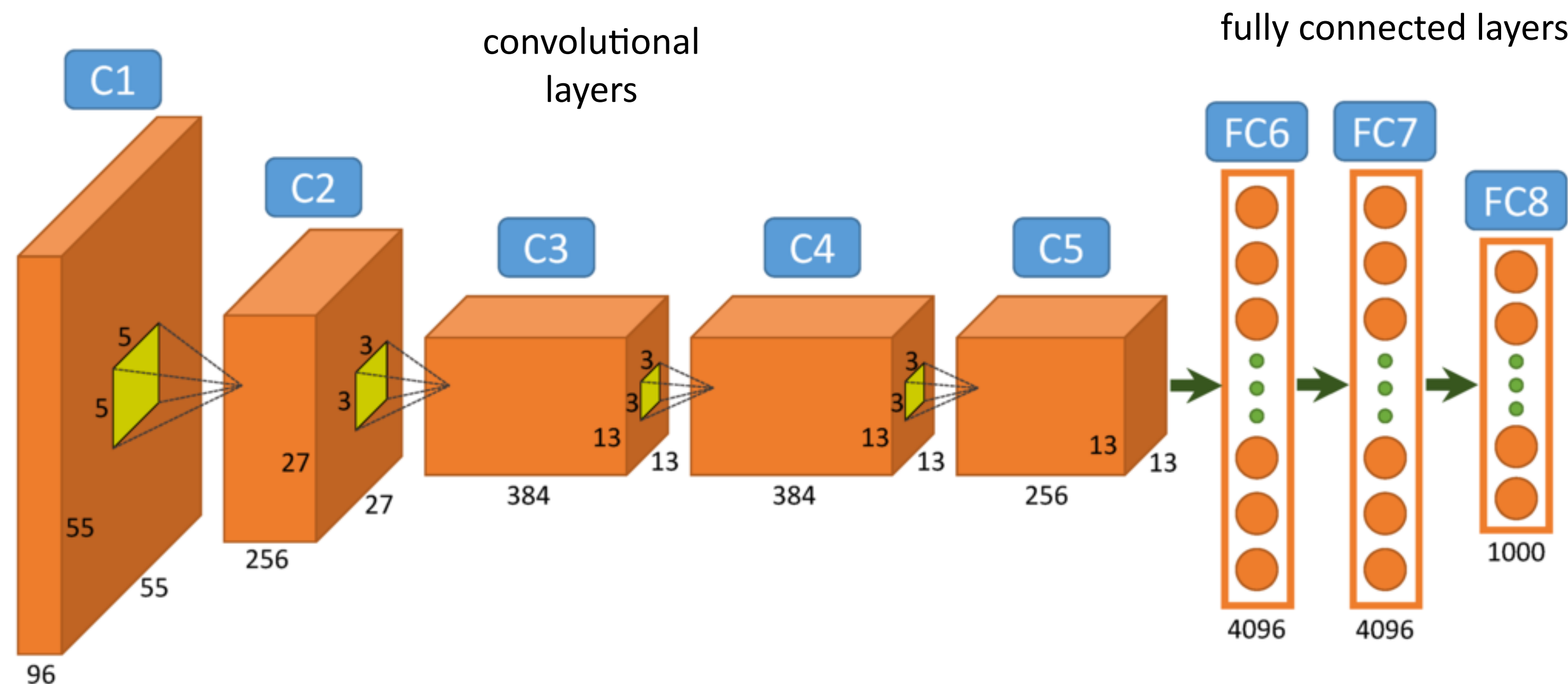
Russakovsky et al. (2012)

ImageNet - Object Recognition



Convolutional Neural Networks

- ▶ AlexNet - one of the first strong results
- ▶ more filters per layer as well as stacked convolutional layers
- ▶ use of ReLU for the non-linear part instead of Sigmoid or Tanh

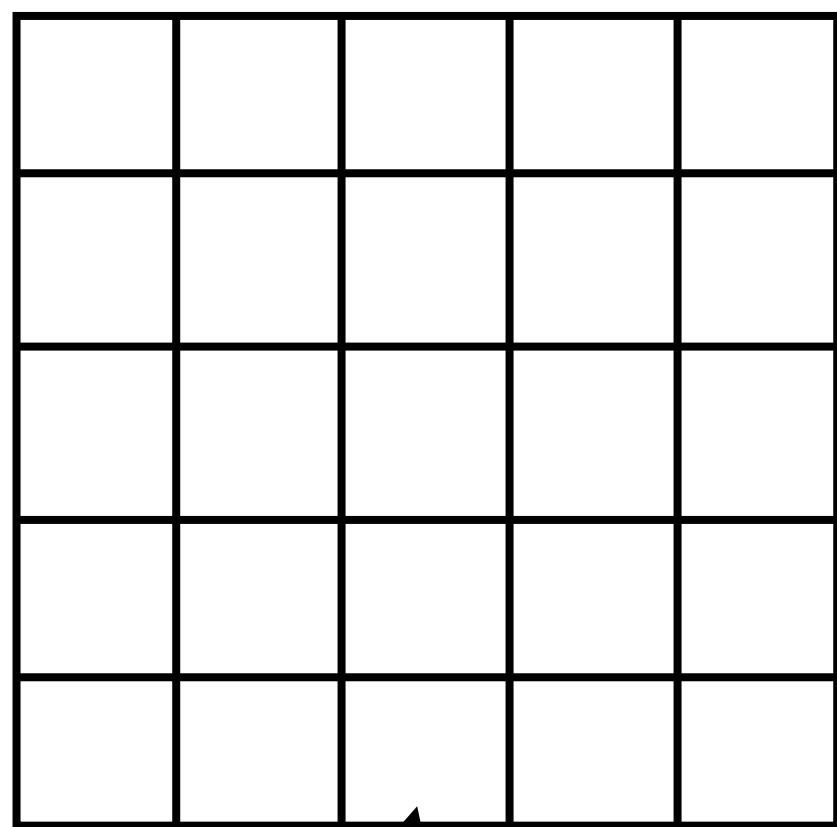


Krizhevsky et al. (2012)

Convolutional Layer

- ▶ Applies a *filter* over patches of the input and returns that filter's activations
- ▶ Convolution: take dot product of filter with a patch of the input

image: $n \times n \times k$ filter: $m \times m \times k$



sum over dot products

$$\text{activation}_{ij} = \sum_{i_o=0}^{m-1} \sum_{j_o=0}^{m-1} \text{image}(i + i_o, j + j_o) \cdot \text{filter}(i_o, j_o)$$

offsets

Each of these cells is a vector with multiple values
Images: RGB values ($k=3$ dim)

Convolutional Layer

- An animated example: $k = 1$, and a filter of size 3×3 .

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

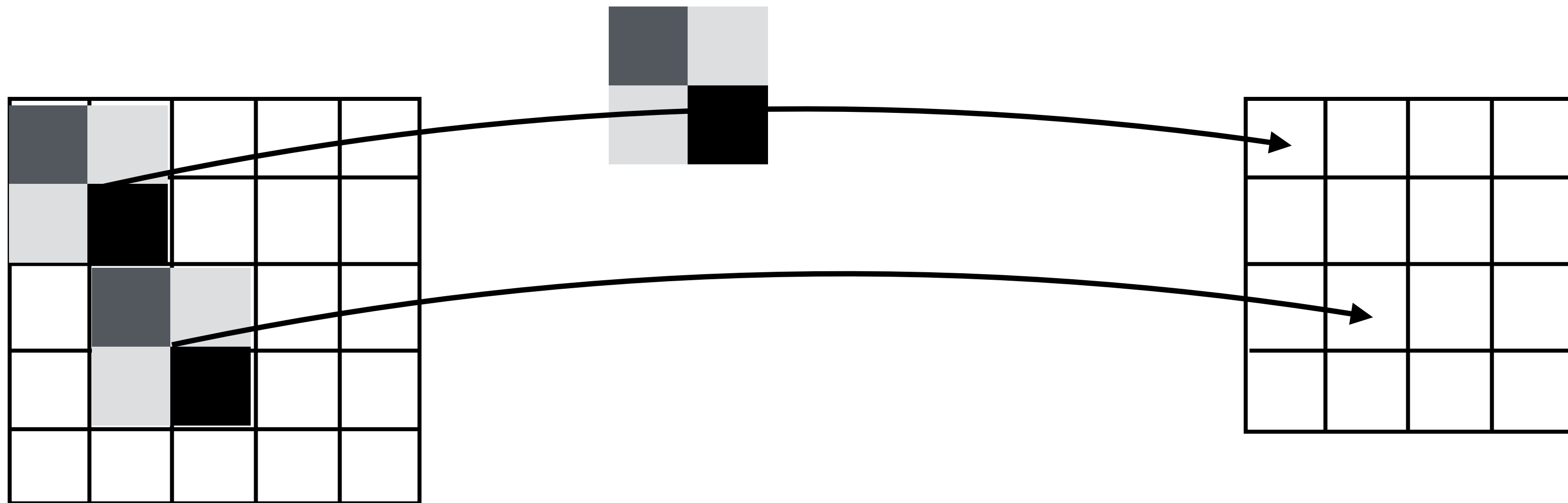
4		

Convolved
Feature

Convolutional Layer

- ▶ Applies a *filter* over patches of the input and returns that filter's activations
- ▶ Convolution: take dot product of filter with a patch of the input

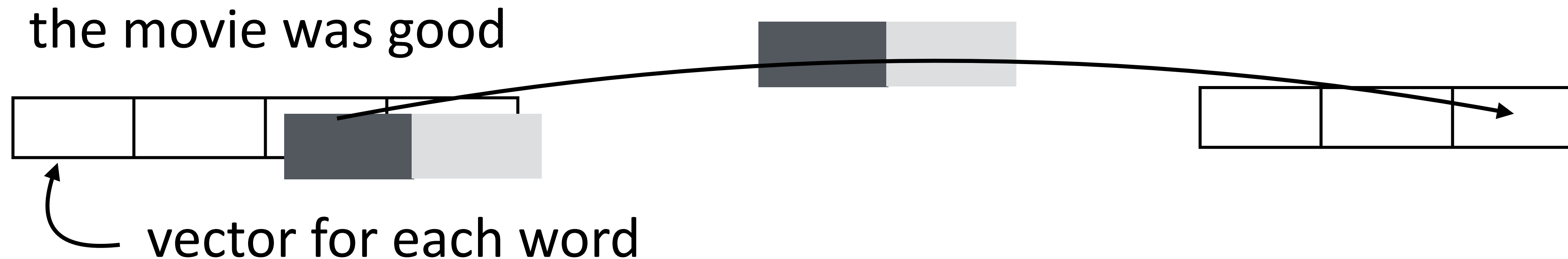
image: $n \times n \times k$ filter: $m \times m \times k$ activations: $(n - m + 1) \times (n - m + 1) \times 1$



Convolutions for NLP

- Input and filter are 2-dimensional instead of 3-dimensional

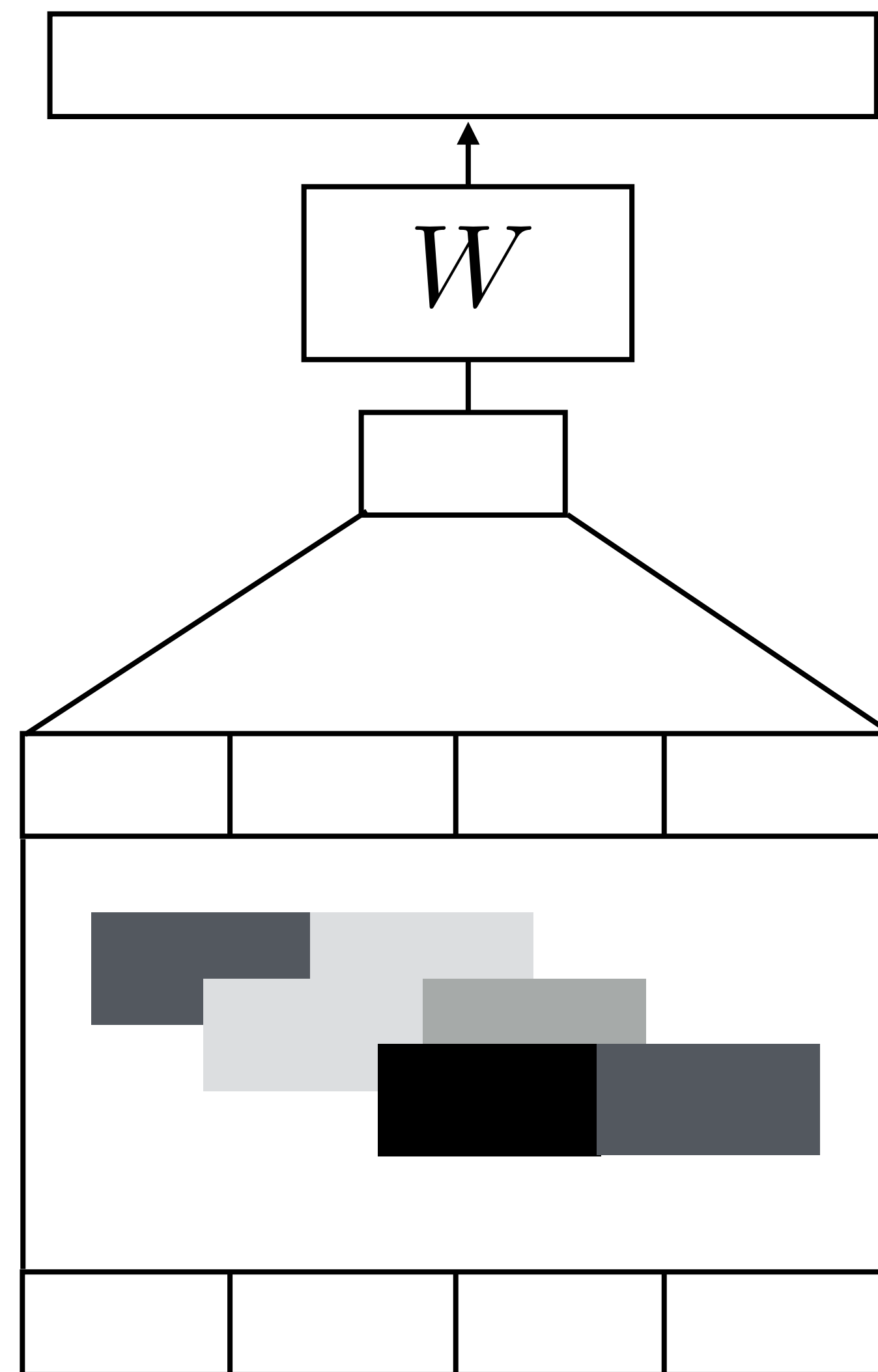
sentence: n words $\times$ k vec dim filter: $m \times k$ activations: $(n - m + 1) \times 1$



- Combines evidence locally in a sentence and produces a new (but still variable-length) representation

CNNs for Sentiment

CNNs for Sentiment Analysis



$$P(y|\mathbf{x})$$

projection + softmax

c-dimensional vector

max pooling over the sentence

$n \times c$

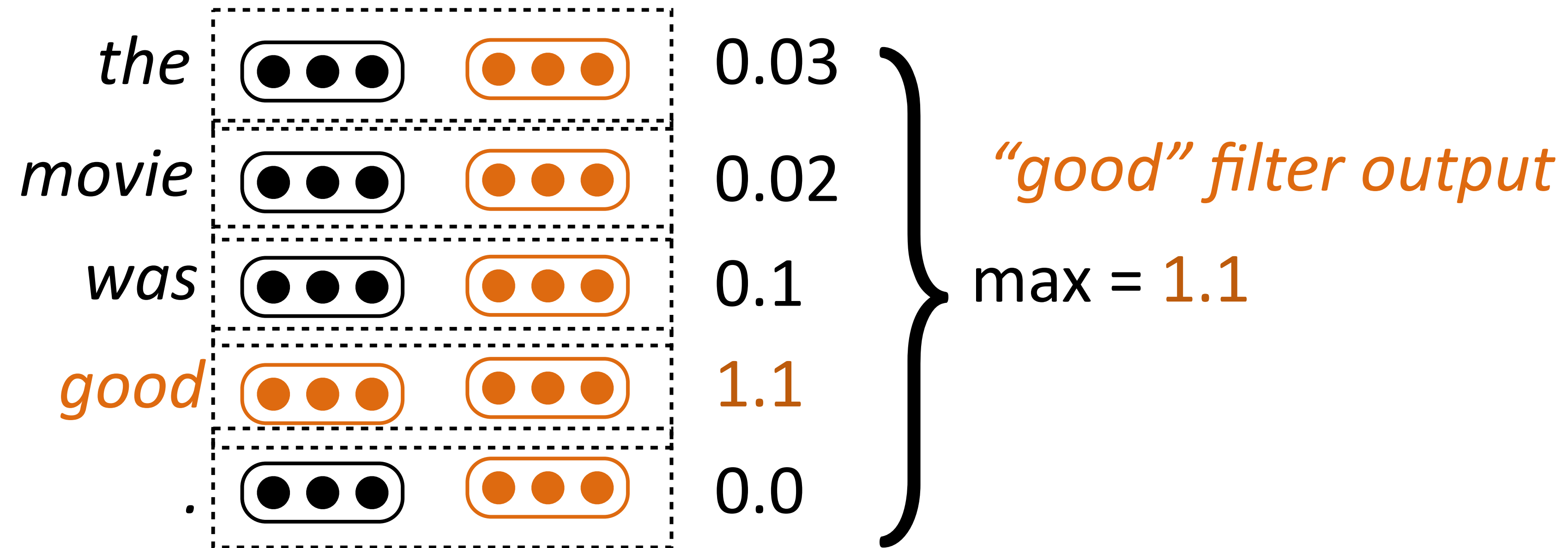
c filters,
 $m \times k$ each

$n \times k$

- ▶ Max pooling: return the max activation of a given filter over the entire sentence; like a logical OR (sum pooling is like logical AND)

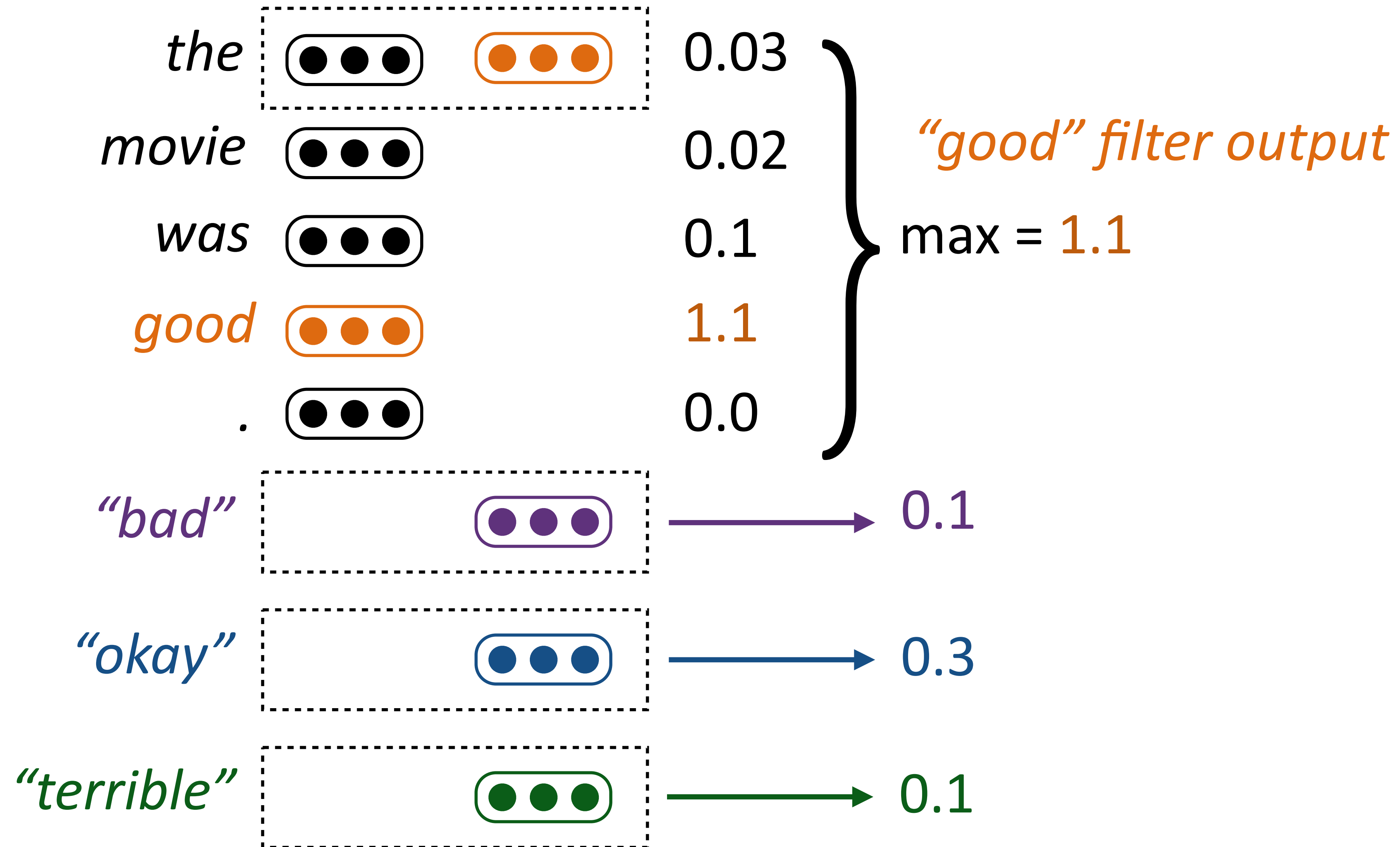
the movie was good

Understanding CNNs for Sentiment

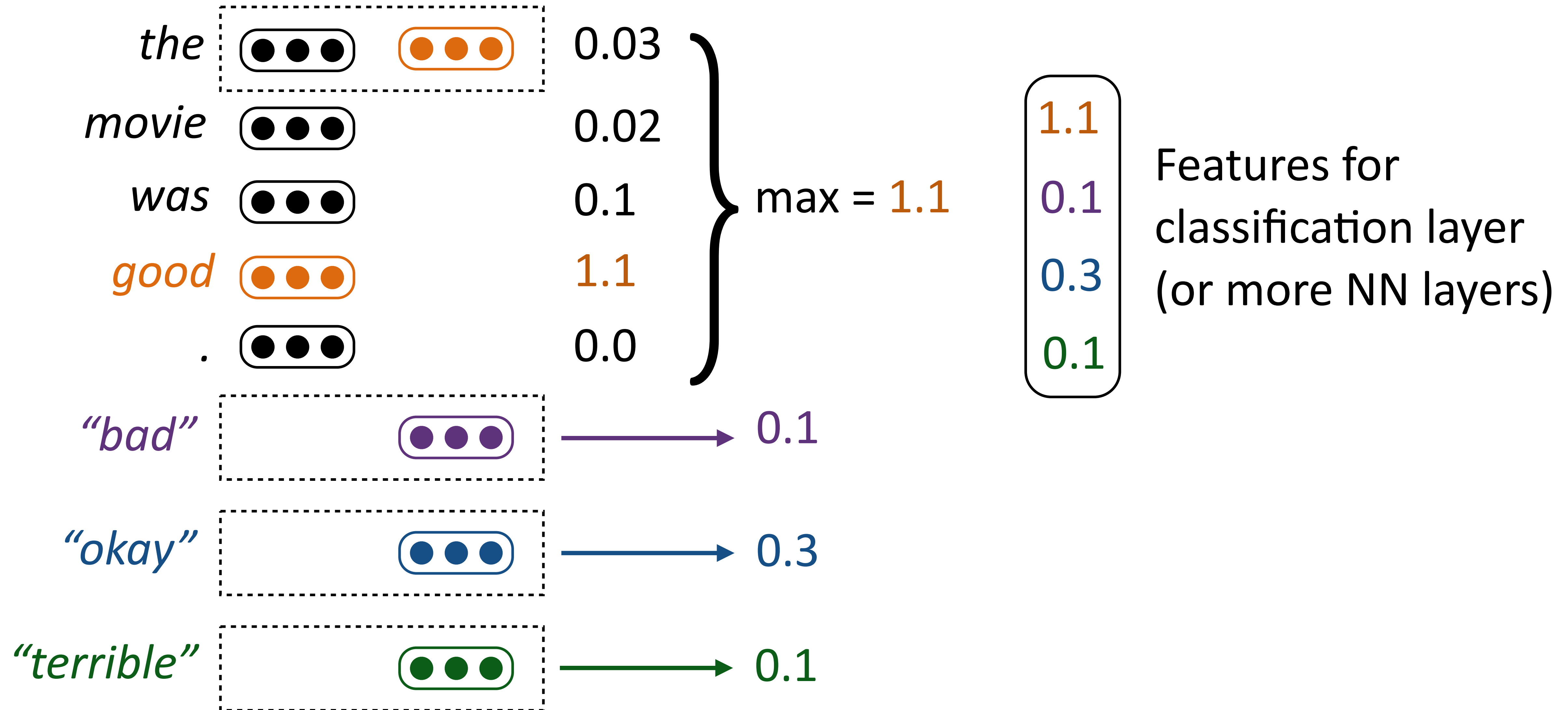


- Filter "looks like" the things that will cause it to have high activation

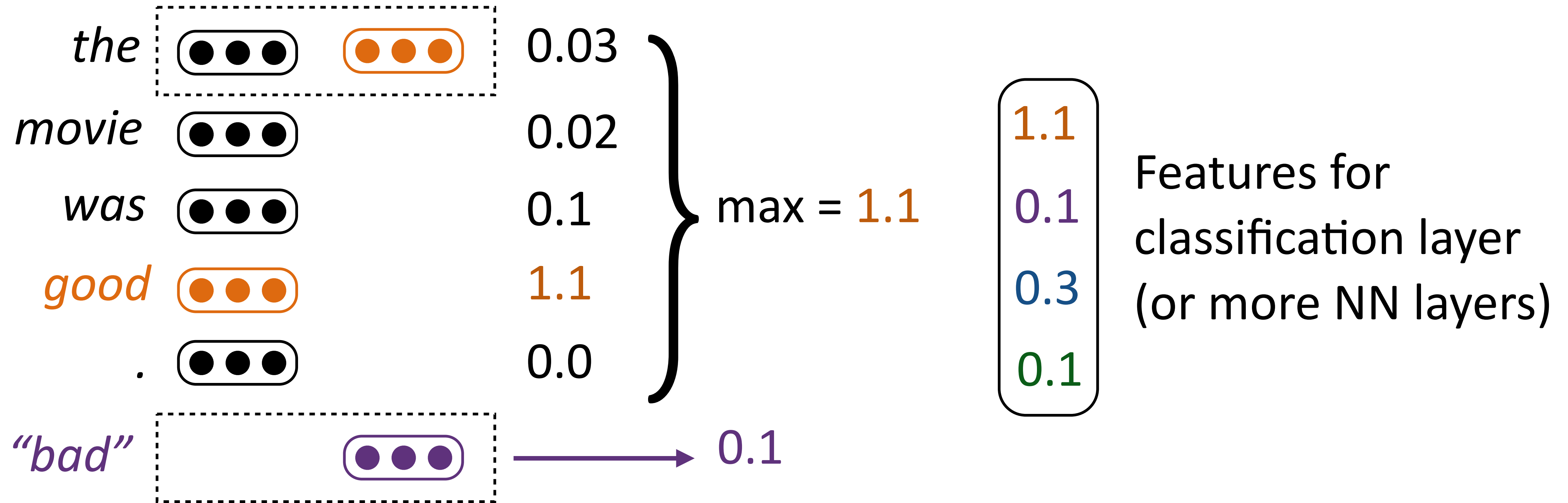
Understanding CNNs for Sentiment



Understanding CNNs for Sentiment

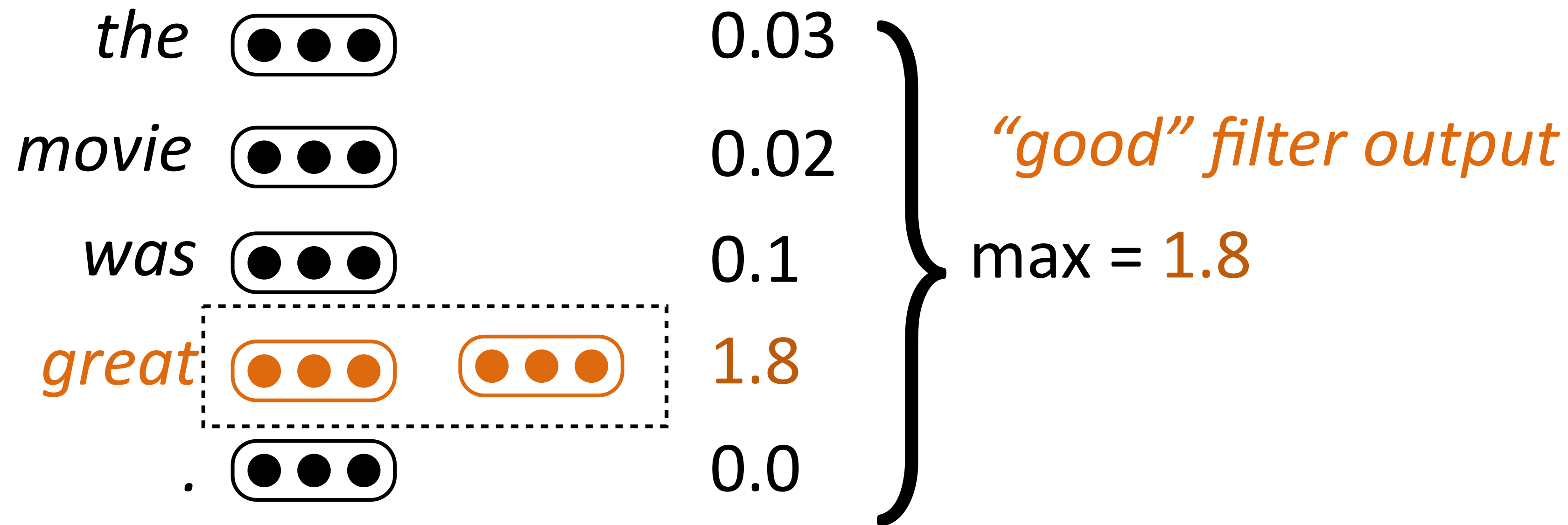


Understanding CNNs for Sentiment



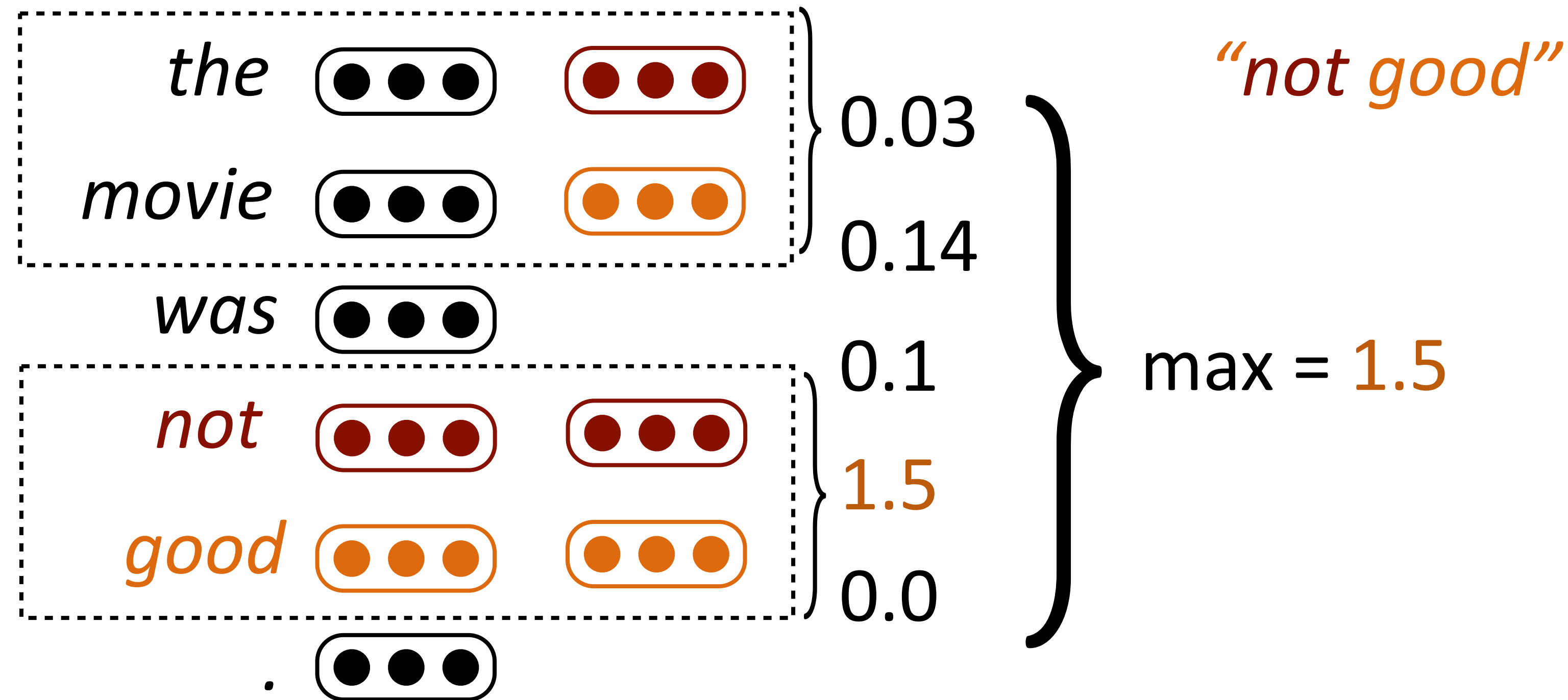
- ▶ Takes variable-length input and turns it into fixed-length output
- ▶ Filters are initialized randomly and then learned

Understanding CNNs for Sentiment



- ▶ Word vectors for similar words are similar, so convolutional filters will have similar outputs

Understanding CNNs for Sentiment



- ▶ Analogous to bigram features in bag-of-words models
- ▶ Indicator feature of text containing bigram $\leftrightarrow$ max pooling of a filter that matches that bigram

What can CNNs learn?

- ▶ CNNs let us take advantage of word similarity

really not very good vs. *really not very enjoyable*

- ▶ CNNs are translation-invariant like bag-of-words

The movie was bad, but blah blah blah ... vs. *... blah blah blah, but the movie was bad.*

- ▶ CNNs can capture local interactions with filters of width > 1

It was not good, it was actually quite bad vs. *it was not bad, it was actually quite good*

CNNs: Implementation

- ▶ Input is batch_size x n x k matrix, filters are c x m x k matrix (c filters)
- ▶ Typically use filters with m ranging from 1 to 5 or so (multiple filter widths in a single convnet)
- ▶ All computation graph libraries support efficient convolution operations

```
CLASS torch.nn.Conv1d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True, padding_mode='zeros')
```

[\[SOURCE\]](#)

Applies a 1D convolution over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C_{in}, L) and output $(N, C_{\text{out}}, L_{\text{out}})$ can be precisely described as:

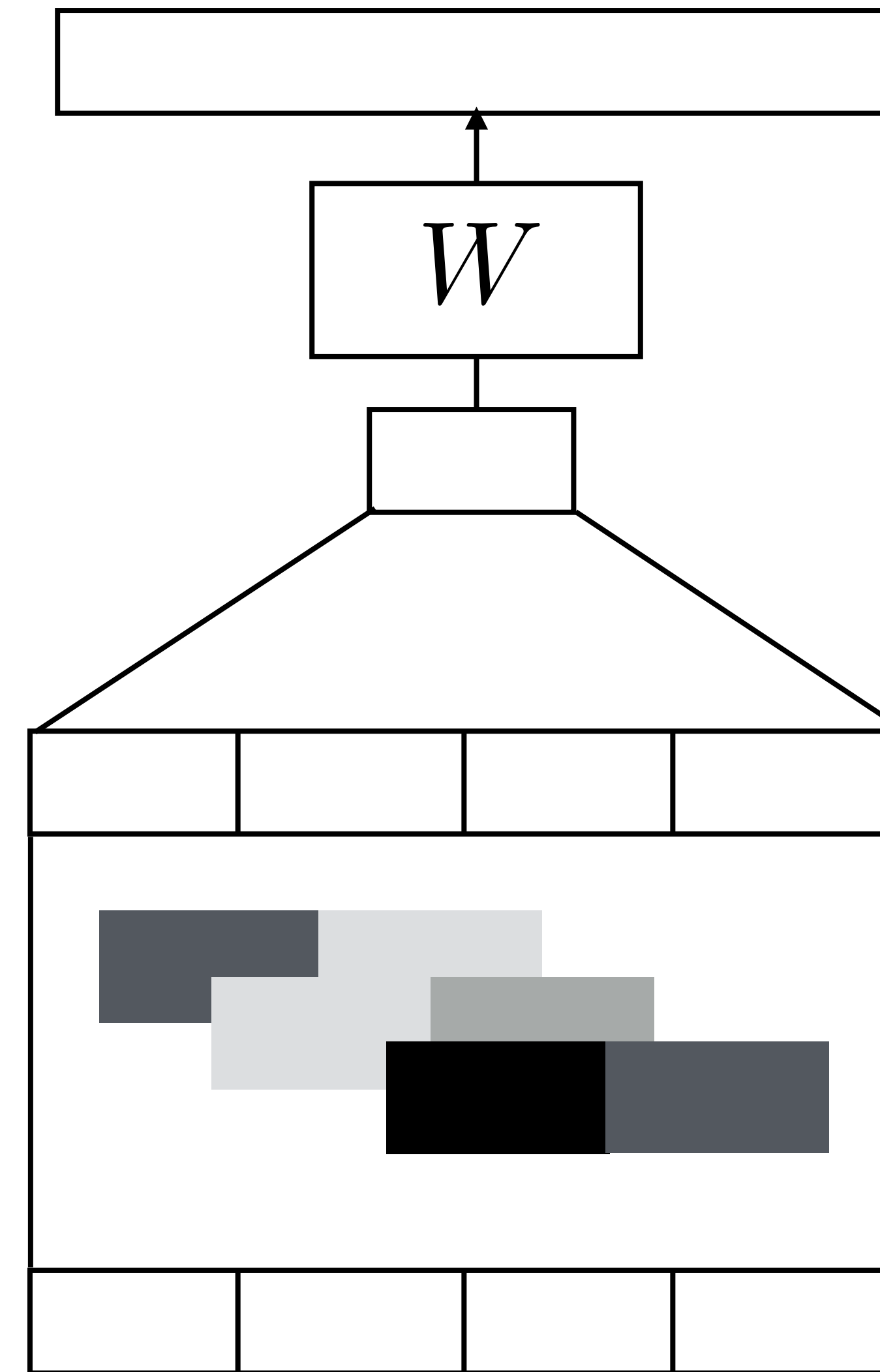
$$\text{out}(N_i, C_{\text{out}_j}) = \text{bias}(C_{\text{out}_j}) + \sum_{k=0}^{C_{\text{in}}-1} \text{weight}(C_{\text{out}_j}, k) \star \text{input}(N_i, k)$$

where $\star$ is the valid **cross-correlation** operator, N is a batch size, C denotes a number of channels, L is a length of signal sequence.

- `stride` controls the stride for the cross-correlation, a single number or a one-element tuple.
- `padding` controls the amount of implicit zero-paddings on both sides for `padding` number of points.

CNNs for Sentence Classification

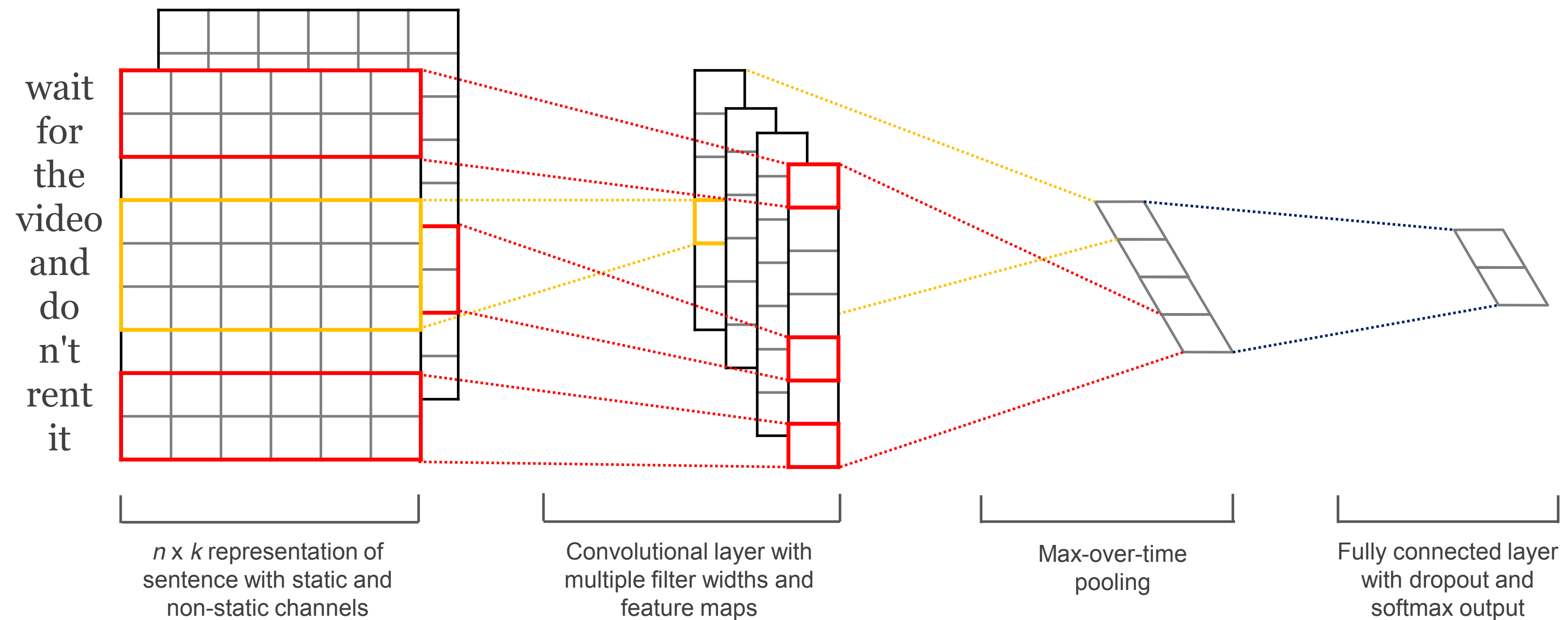
- ▶ Question classification, sentiment, etc.
- ▶ Conv+pool, then use feedforward layers to classify
- ▶ Can use multiple types of input vectors (fixed initializer and learned)



the movie was good

Kim (2014)

CNNs for Sentence Classification



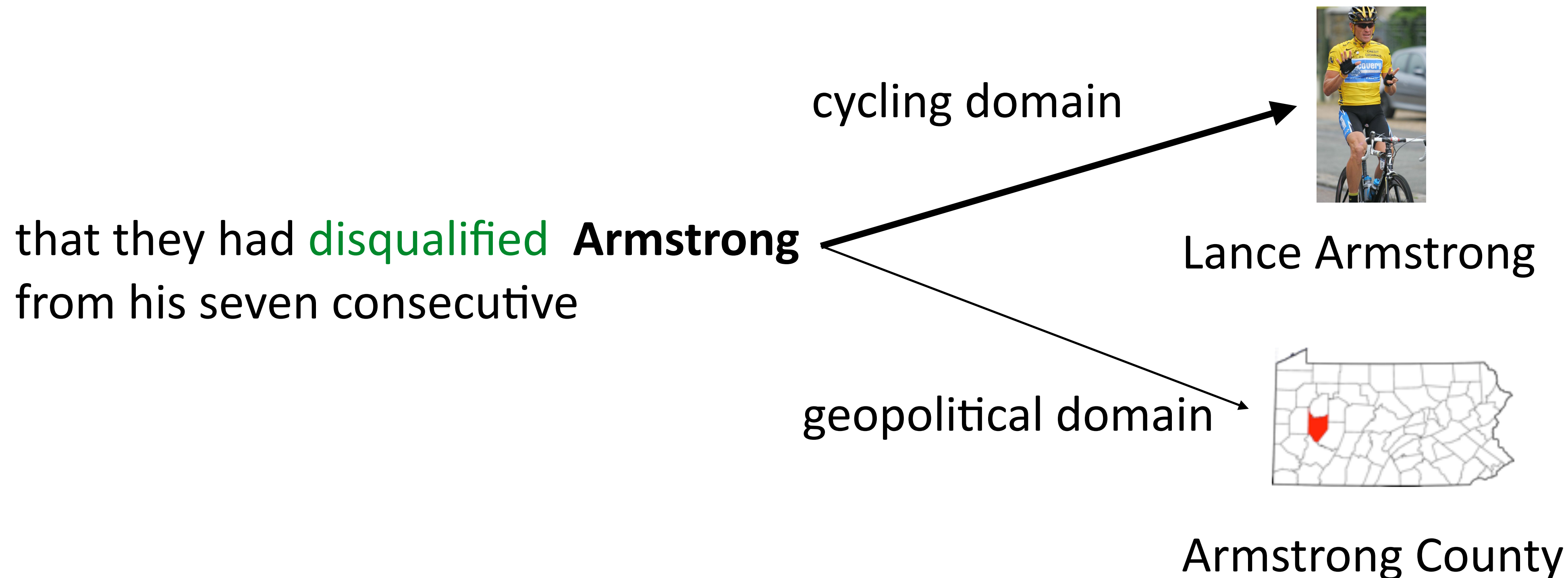
Sentence Classification

Model	MR	SST-1	SST-2	Subj	TREC	CR	MPQA
CNN-multichannel	81.1	47.4	88.1	93.2	92.2	85.0	89.4
NBSVM (Wang and Manning, 2012)	79.4	—	—	93.2	—	81.8	86.3

- ▶ Also effective at document-level text classification

Entity Linking

- ▶ CNNs can produce good representations of both sentences and documents like typical bag-of-words features
- ▶ Can distill topic representations for use in entity linking



Entity Linking

Although he originally won the event, the United States Anti-Doping Agency announced in August 2012 that they had disqualified **Armstrong** from his seven consecutive Tour de France wins from 1999–2005.



Lance Edward Armstrong is an American former professional road cyclist



Armstrong County is a county in Pennsylvania...

CNN

CNN

CNN

Document topic vector d

Article topic vector a_{Lance}

Article topic vector

a_{County}

$$s_{\text{Lance}} = d \cdot a_{\text{Lance}}$$

$$s_{\text{County}} = d \cdot a_{\text{County}}$$

$$P(y|\mathbf{x}) = \text{softmax}(\mathbf{s})$$

Francis-Landau et al. (2016)

Entity Linking

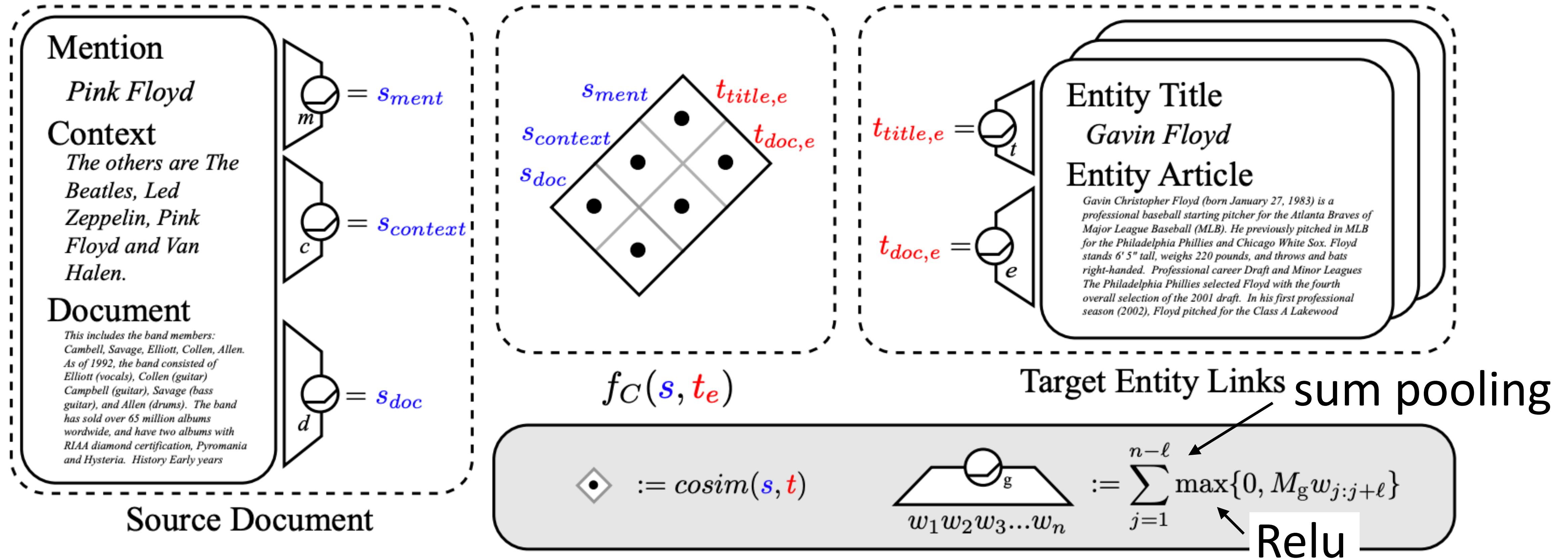
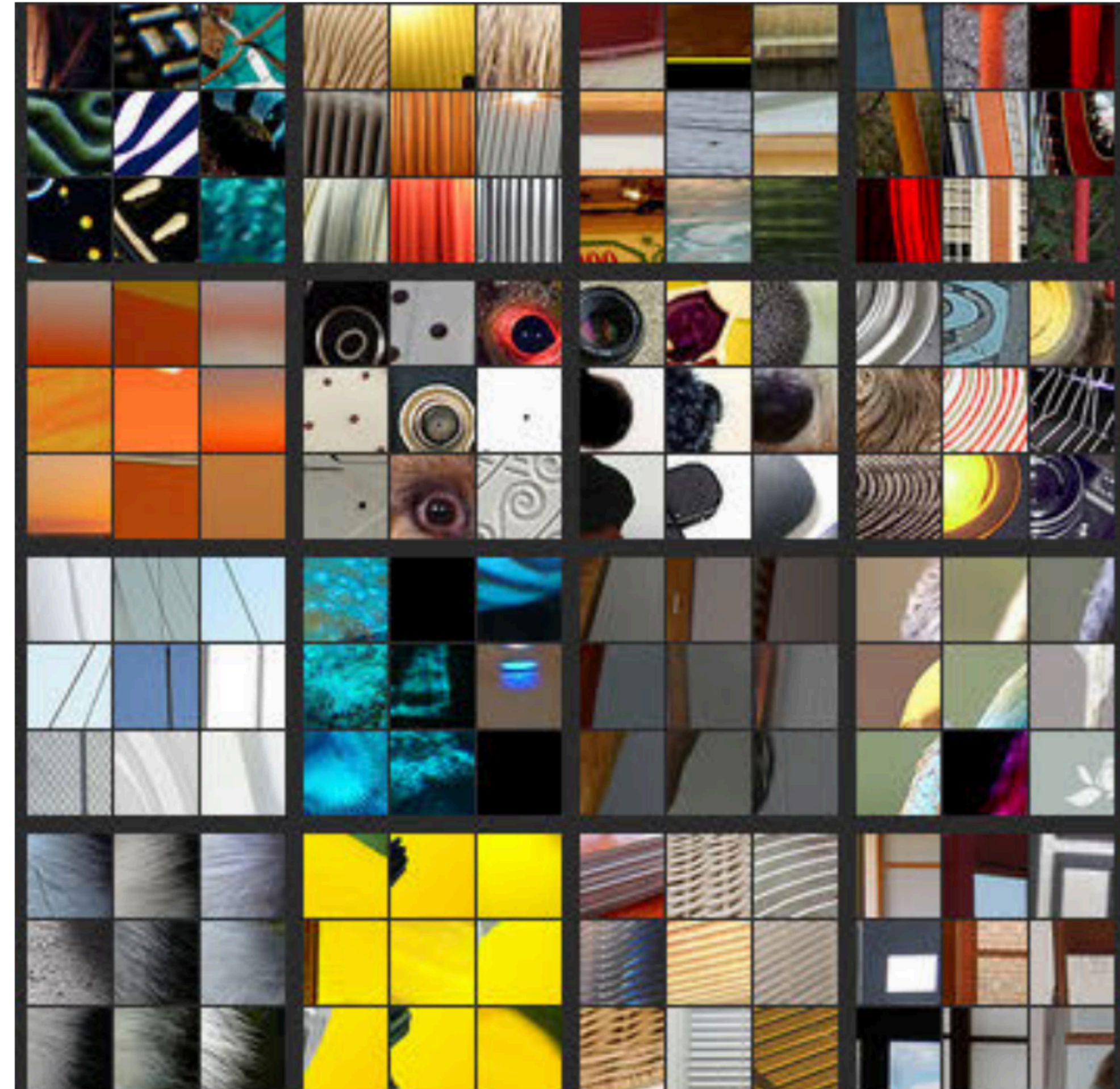
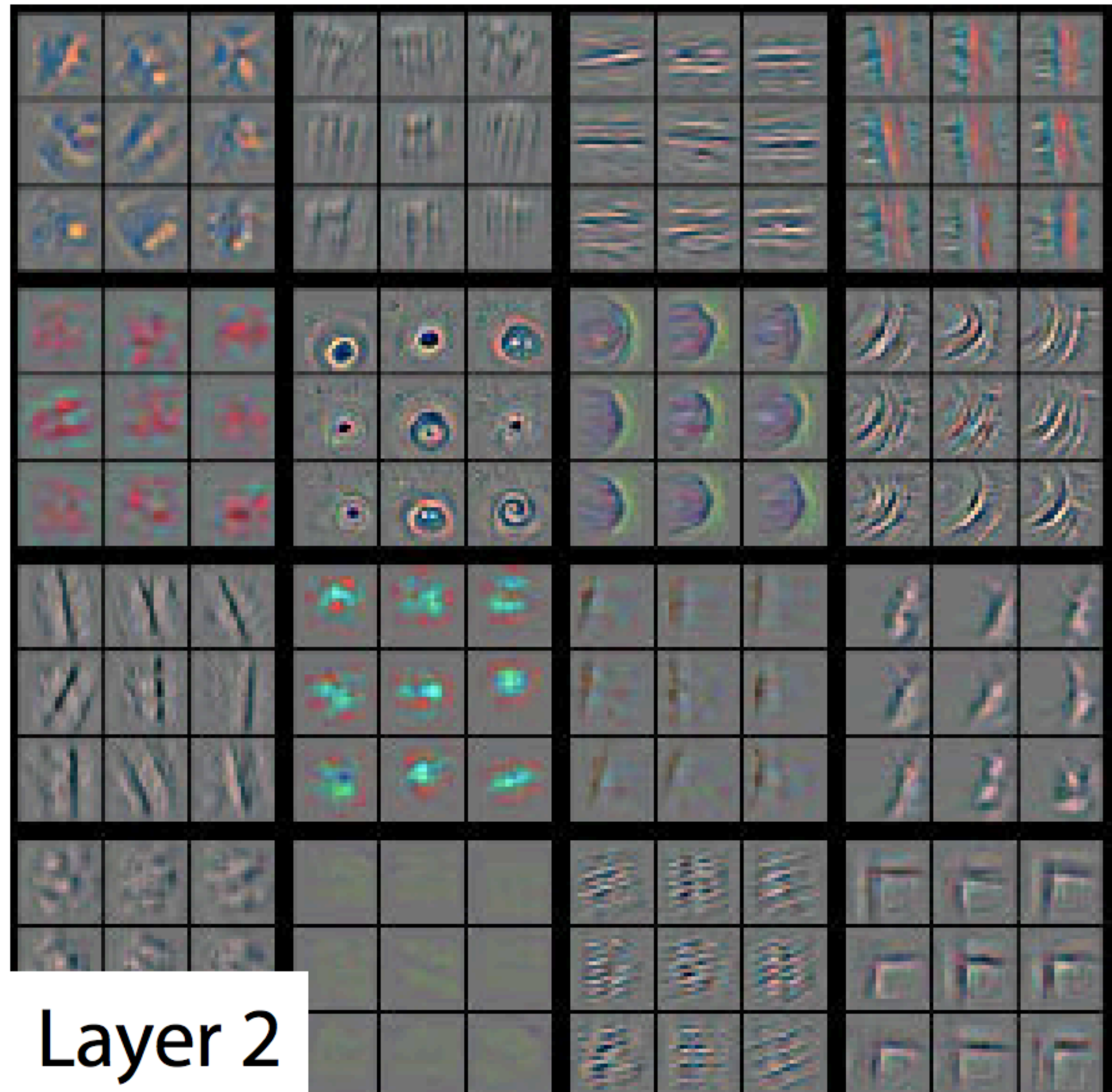


Figure 1: Extraction of convolutional vector space features $f_C(x, t_e)$. Three types of information from the input document and two types of information from the proposed title are fed through convolutional networks to produce vectors, which are systematically compared with cosine similarity to derive real-valued semantic similarity features.

Deep Convolutional Networks

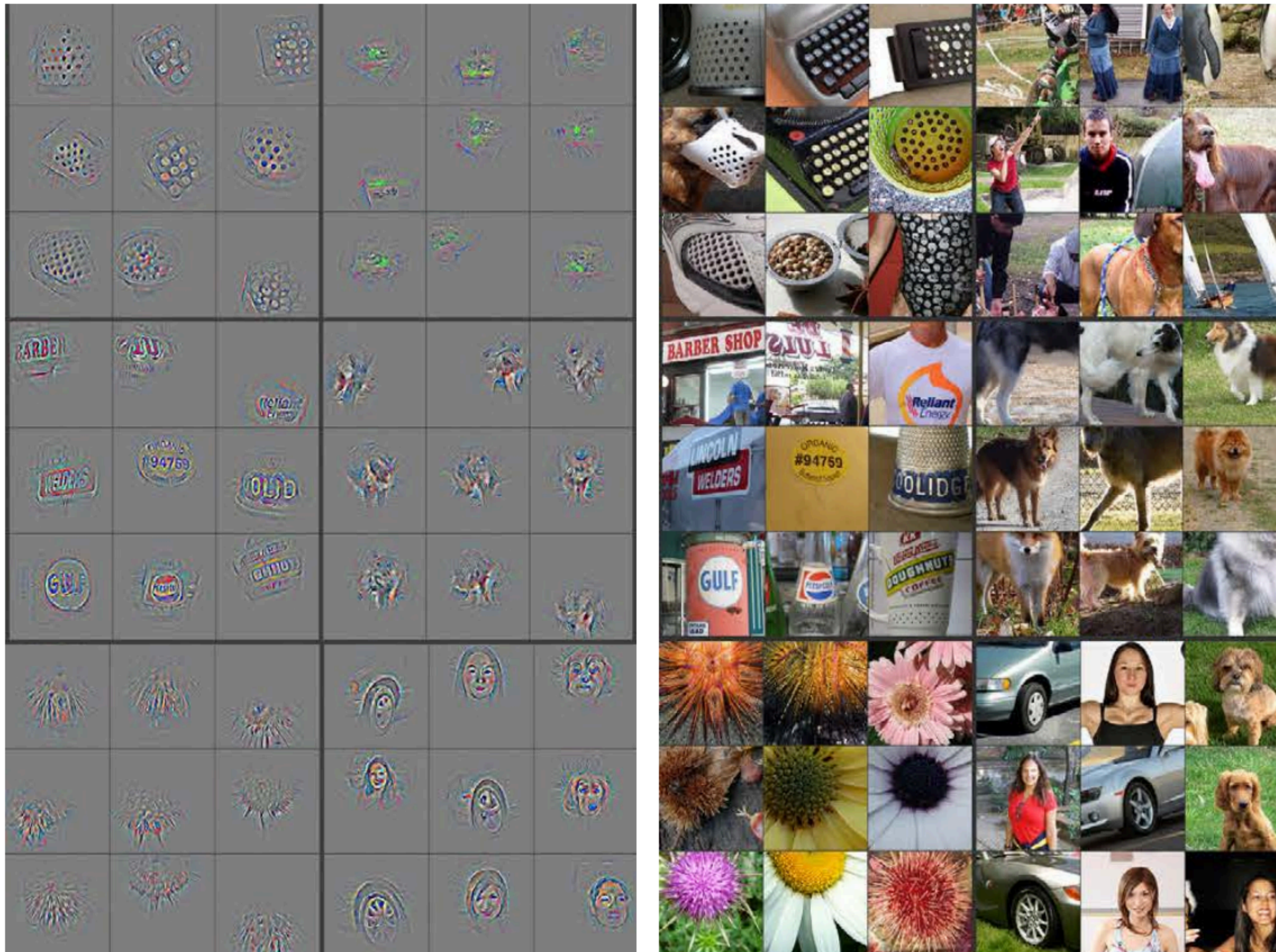
- ▶ Low-level filters: extract low-level features from the data



Zeiler and Fergus (2014)

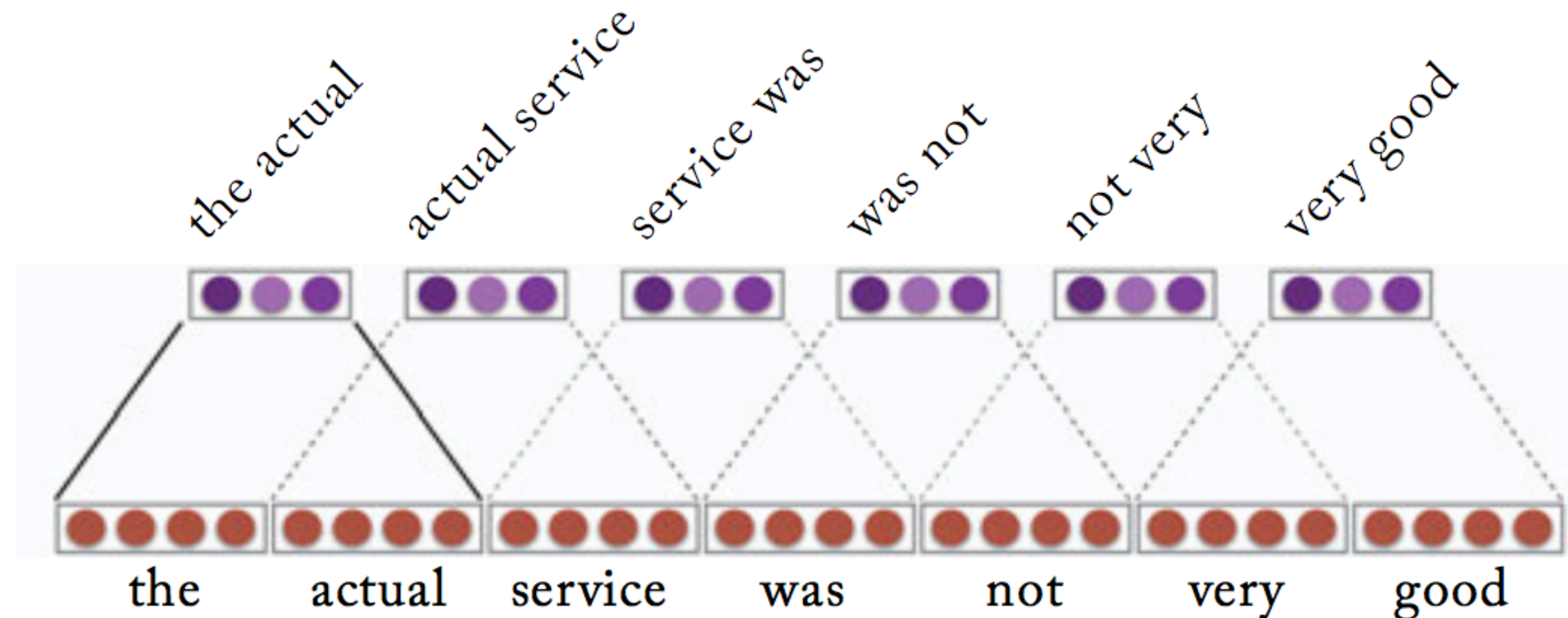
Deep Convolutional Networks

- High-level filters: match larger and more “semantic patterns”



Zeiler and Fergus (2014)

Multi-layer CNNs for Language



- ▶ CNNs are a flexible way of extracting features analogous to bag of n-grams, can also encode positional information